



Recursion

Dr. Obed Noé Sámano Abonce



Recursion

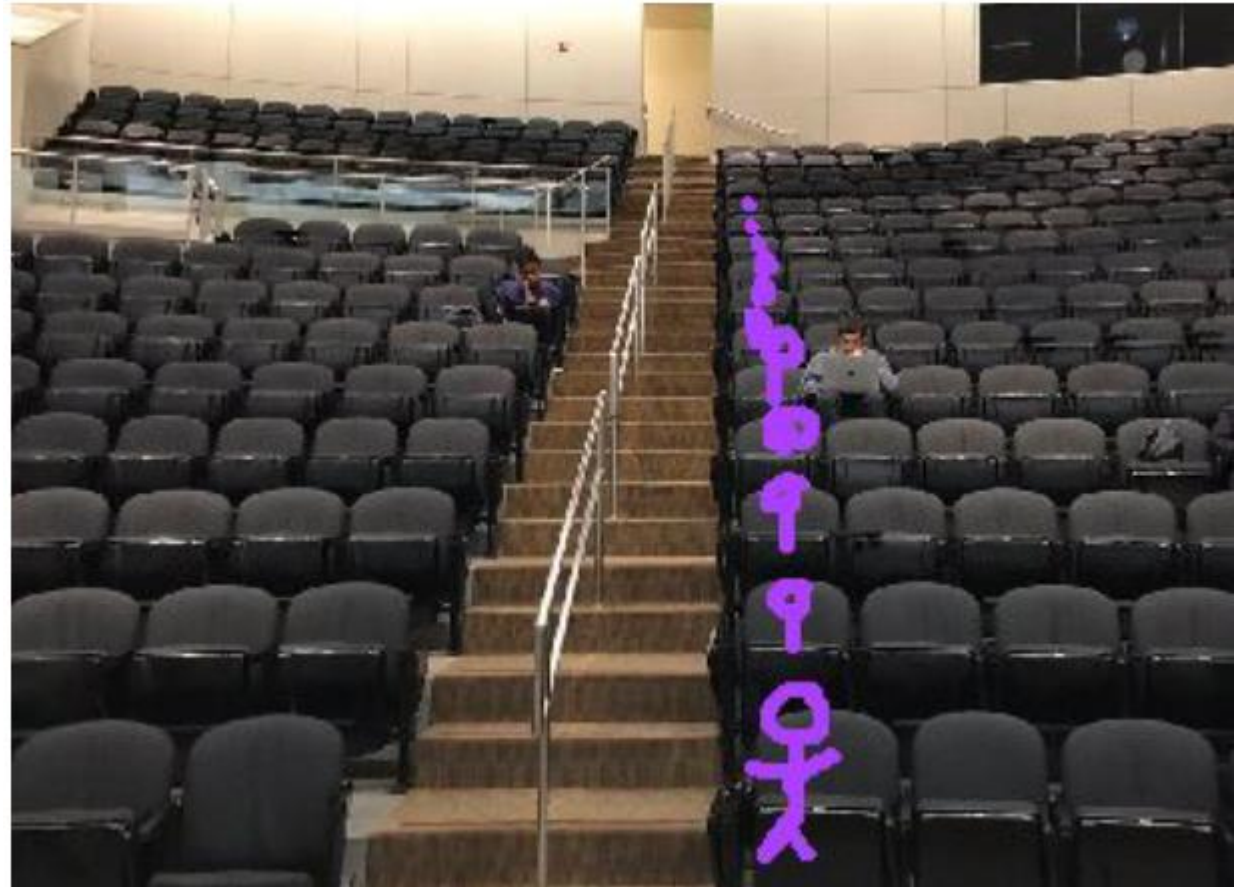
What is recursion?



Recursion

Example

How many
students are in
the lecture hall?



Recursion

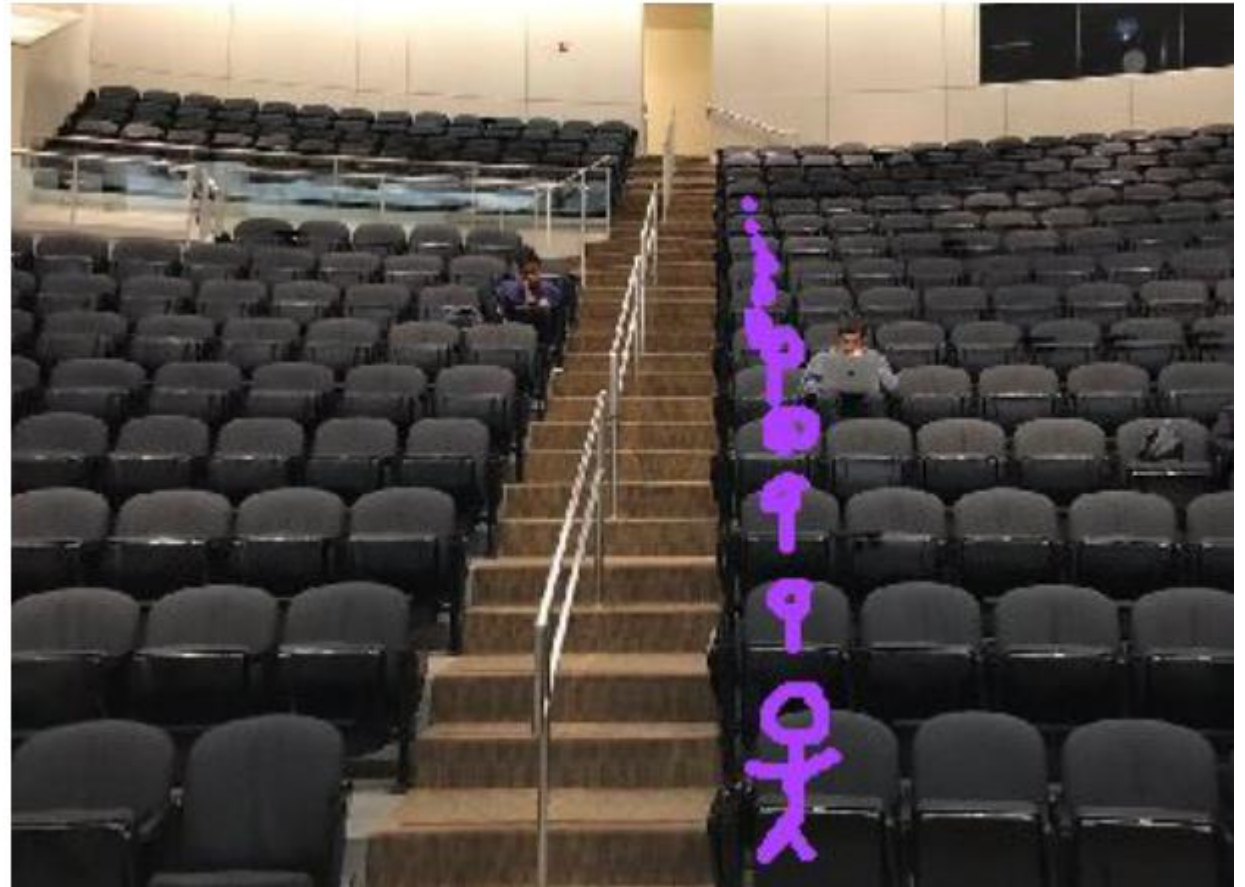
Example

Let's focus on a single column:

I go to the first person in the front row and ask: "How many people are sitting directly behind you in your 'column'?"

Student's algorithm:

- If there is no one behind me, answer 0.
- If someone is sitting behind me:
 - Ask that person: How many people are sitting directly behind you in your "column"?
 - When they respond with a value N , respond $(N + 1)$ to the person who asked me.



Recursion

What is recursion?

Non-recursive function

It is computed in a single step.

Example:

$$f(x) = 2 \cdot x + 1$$

Recursive function

A recursive function is defined in terms of itself.

Example:

$$f(x) = 2 \cdot f(x - 1) + x$$

with $f(0) = 0$

From the definition:

$$f(1)=1 \quad f(2)=4 \quad f(3)=11 \quad f(4)=26$$

Recursion

The two parts of every recursive function

Base case

The value known directly with no recursion.
In the previous example, $f(0) = 0$.

Without a base case the definition is meaningless.

Recursive case

The function calls itself on a smaller input and builds the answer. In the previous example:

$$2 \cdot f(x - 1) + x^2.$$

Quick check: Is there a base case? Does every call move toward it?

Recursion

Coding

```
int f(int x) {  
    if (x == 0)           // base case  
        return 0;  
    else                  // recursive case  
        return 2 * f(x - 1) + x;  
}
```

Recursion

Example: factorial

The number **n factorial**, denoted **n!**, is

$$n \times (n - 1) \times \dots \times 3 \times 2 \times 1$$

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n - 1)! & \text{otherwise} \end{cases}$$

For example,

- $3! = 3 \times 2 \times 1 = 6.$
- $4! = 4 \times 3 \times 2 \times 1 = 24.$
- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120.$
- $0! = 1.$ (by definition)

Recursion in action

```
int main() {  
    int n = factorial(5);  
    cout << "5! = " << n << endl;  
    return 0;  
}
```

Recursion in action

```
int main() {  
    int n = factorial(5);  
    cout << "5! = " << n << endl;  
    return 0;  
}
```

Recursion in action

```
int main() {  
    int factorial (int n) {  
        if (n == 0) {  
            return 1;  
        } else {  
            return n * factorial(n-1);  
        }  
    }  
}
```



When a function gets called, a new stack frame gets created.

A “frame” stores all of the data (variables) for that particular function call.

Recursion in action

```
int main() {  
    int factorial (int n) {  
        if (n == 0) {  
            return 1;  
        } else {  
            return n * factorial(n-1);  
        }  
    }  
}
```



Recursion in action

```
int main() {  
    int factorial (int n) {  
        if (n == 0) {  
            return 1;  
        } else {  
            return n * factorial(n-1);  
        }  
    }  
}
```



Recursion in action

```
int main() {
```

```
}
```

```
int factorial (int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n-1);
```

```
    }
```

```
}
```



n

Recursion in action

```
int main() {  
    int factorial (int n) {  
        if (n == 0) {  
            return 1;  
        } else {  
            return n * factorial(n-1);  
        }  
    }  
}
```



Recursion in action

```
int main() {
```

```
}
```

```
int factorial (int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n-1);
```

```
    }
```

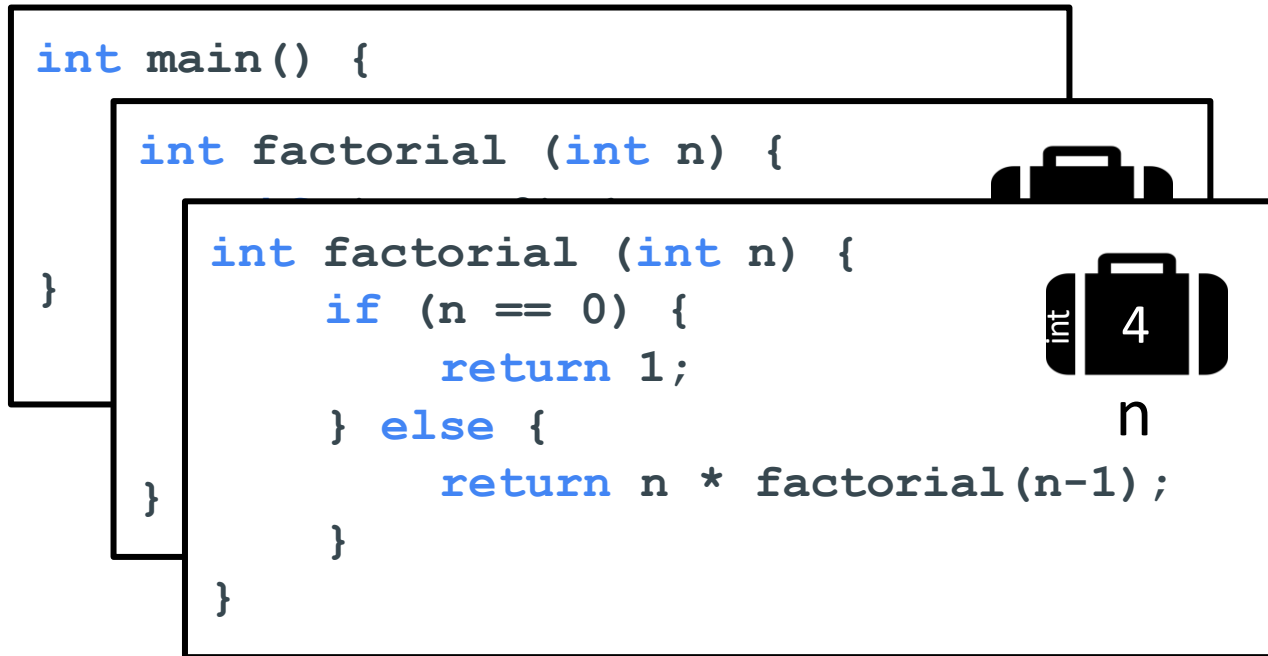
```
}
```



n

5

Recursion in action



Every time we call **factorial()**, we get a new copy of the local variable **n** that's independent of all the previous copies because it exists inside the new frame.

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
}
```

```
    int factorial (int n) {
```

```
        if (n == 0) {
```

```
            return 1;
```

```
        } else {
```

```
            return n * factorial(n-1);
```

```
        }
```

```
    }
```



n

4

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```

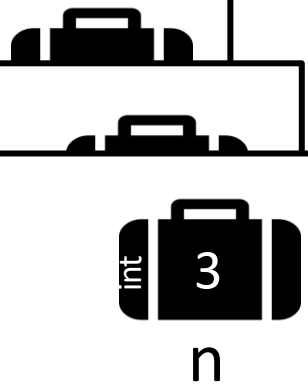


n

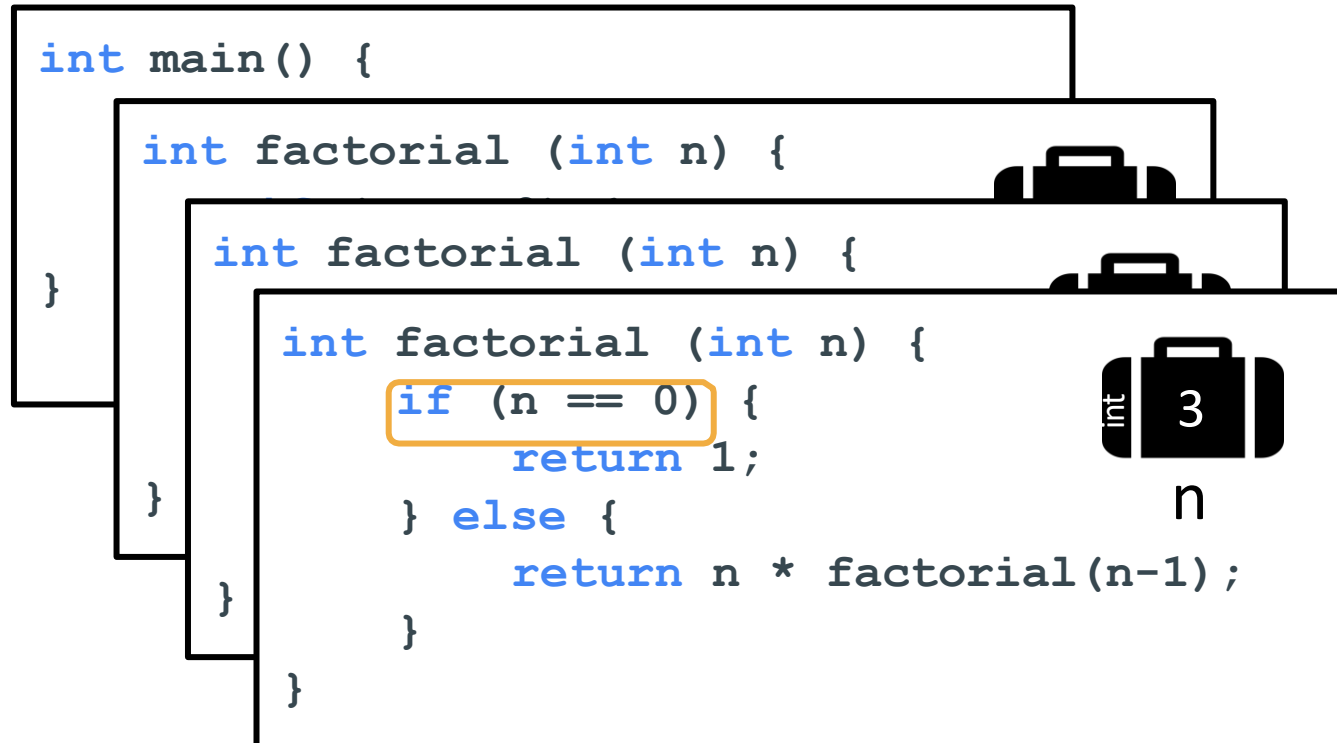
4

Recursion in action

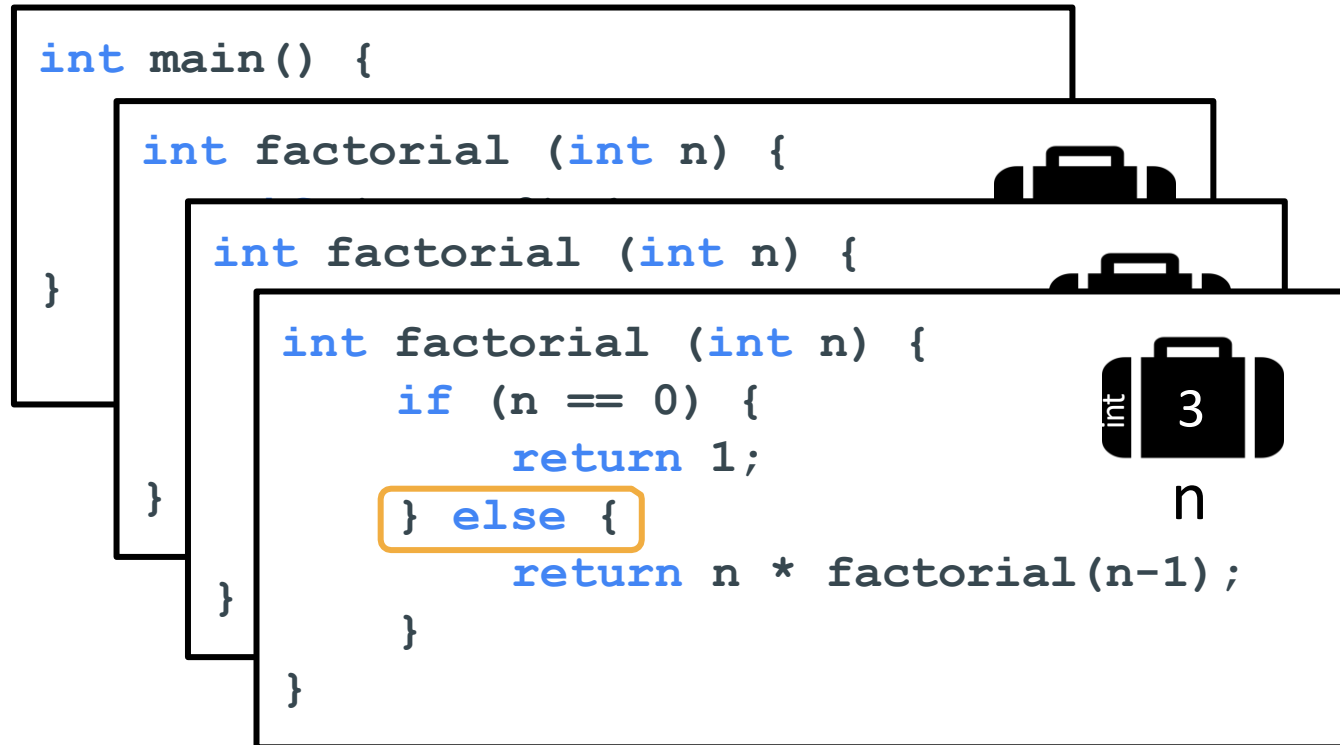
```
int main() {  
    int factorial (int n) {  
        int factorial (int n) {  
            int factorial (int n) {  
                if (n == 0) {  
                    return 1;  
                } else {  
                    return n * factorial(n-1);  
                }  
            }  
        }  
    }  
}
```



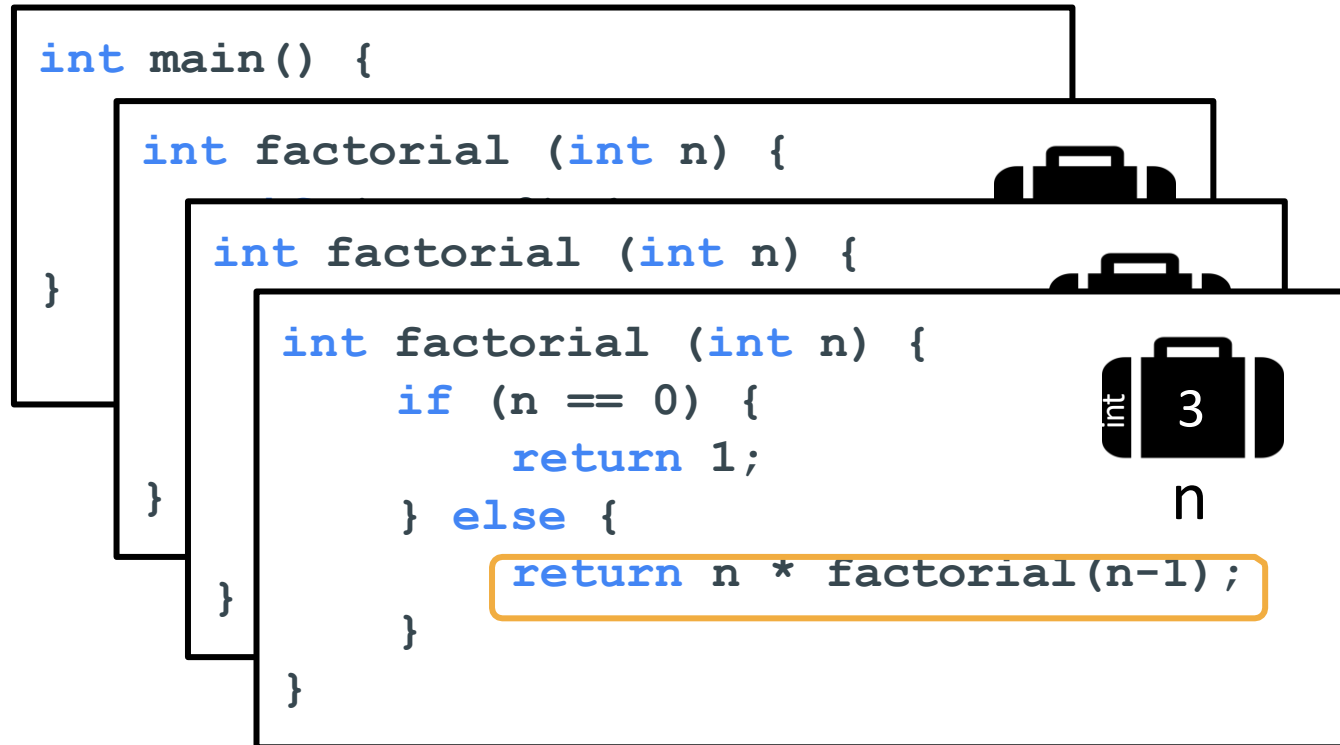
Recursion in action



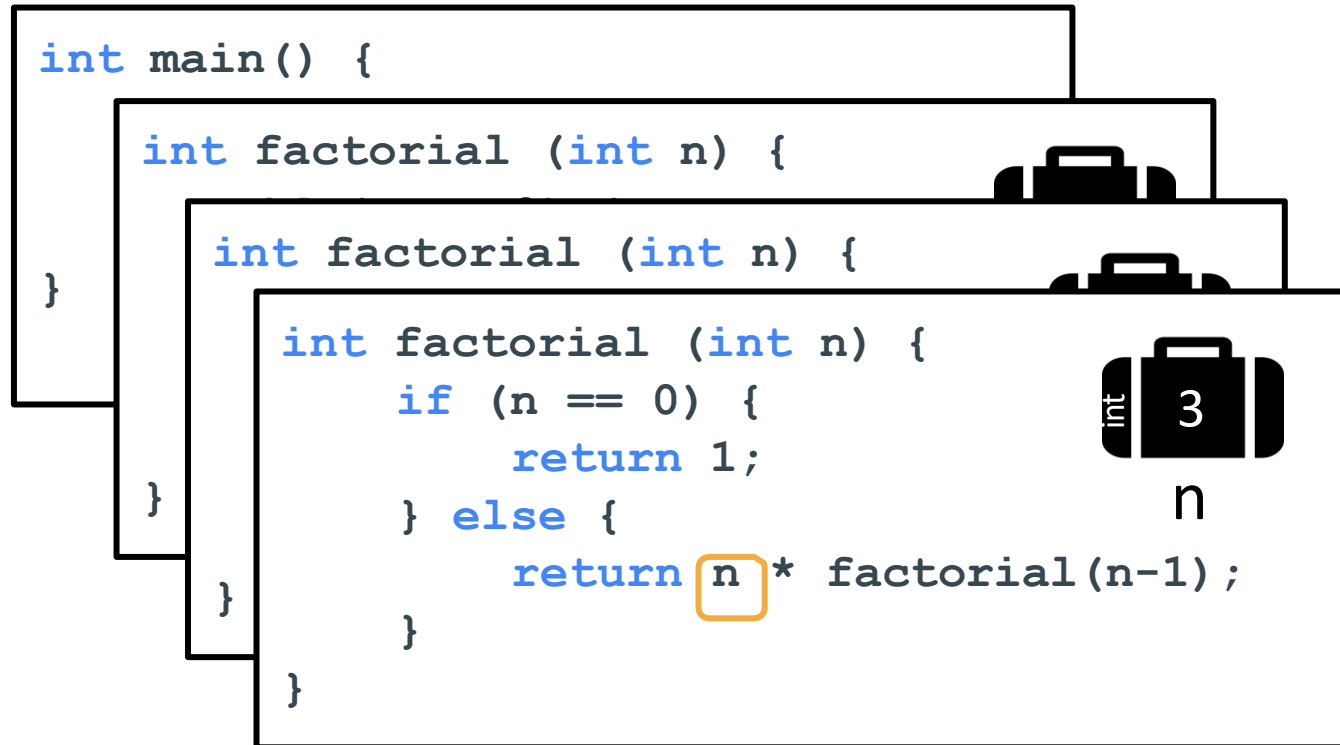
Recursion in action



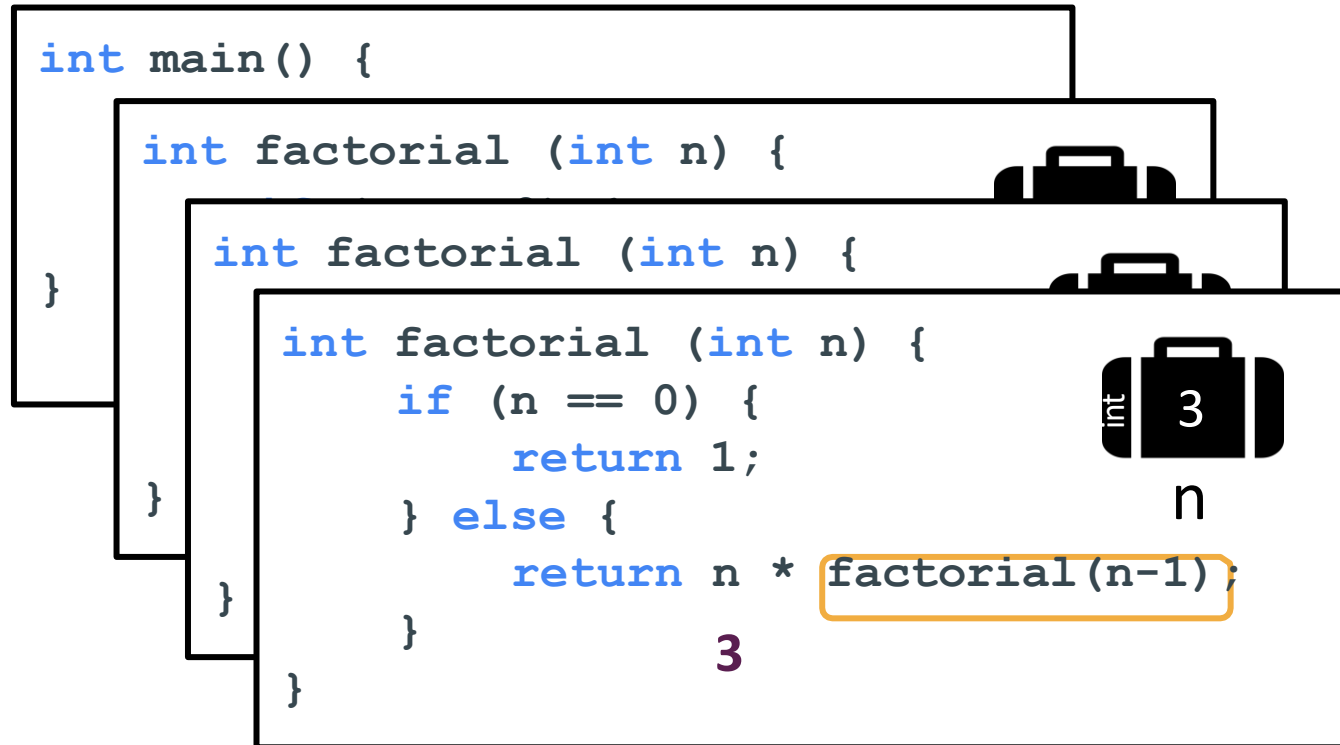
Recursion in action



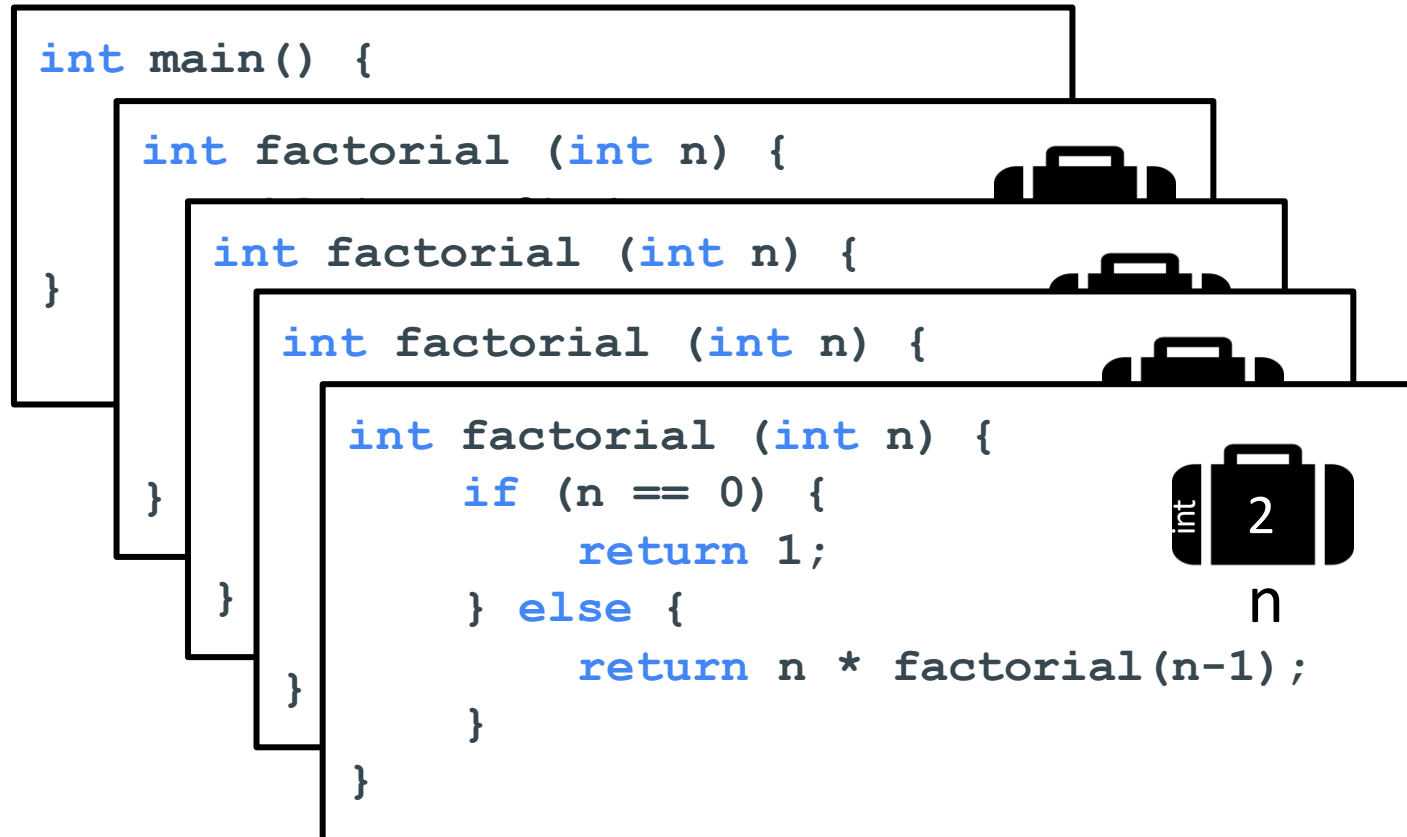
Recursion in action



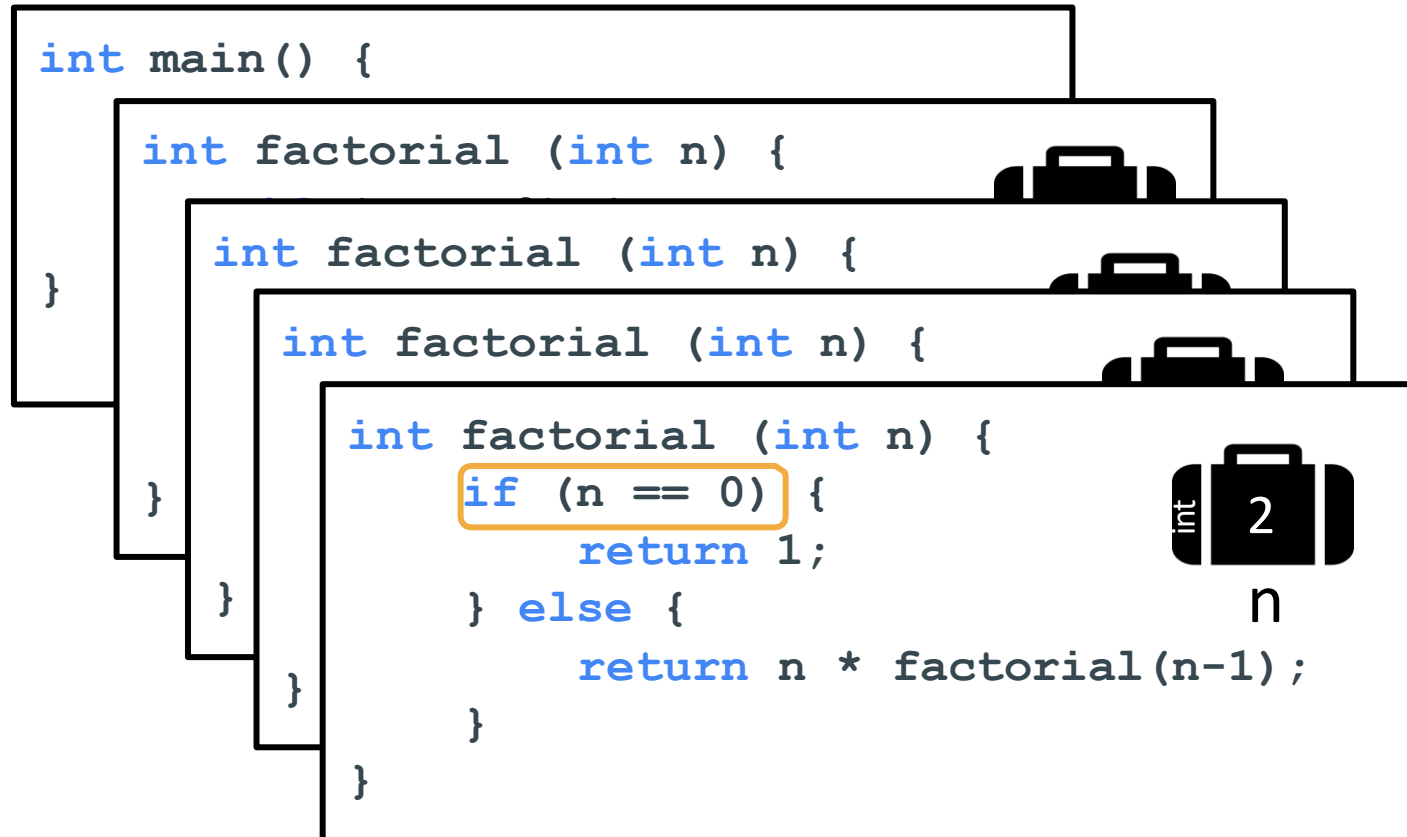
Recursion in action



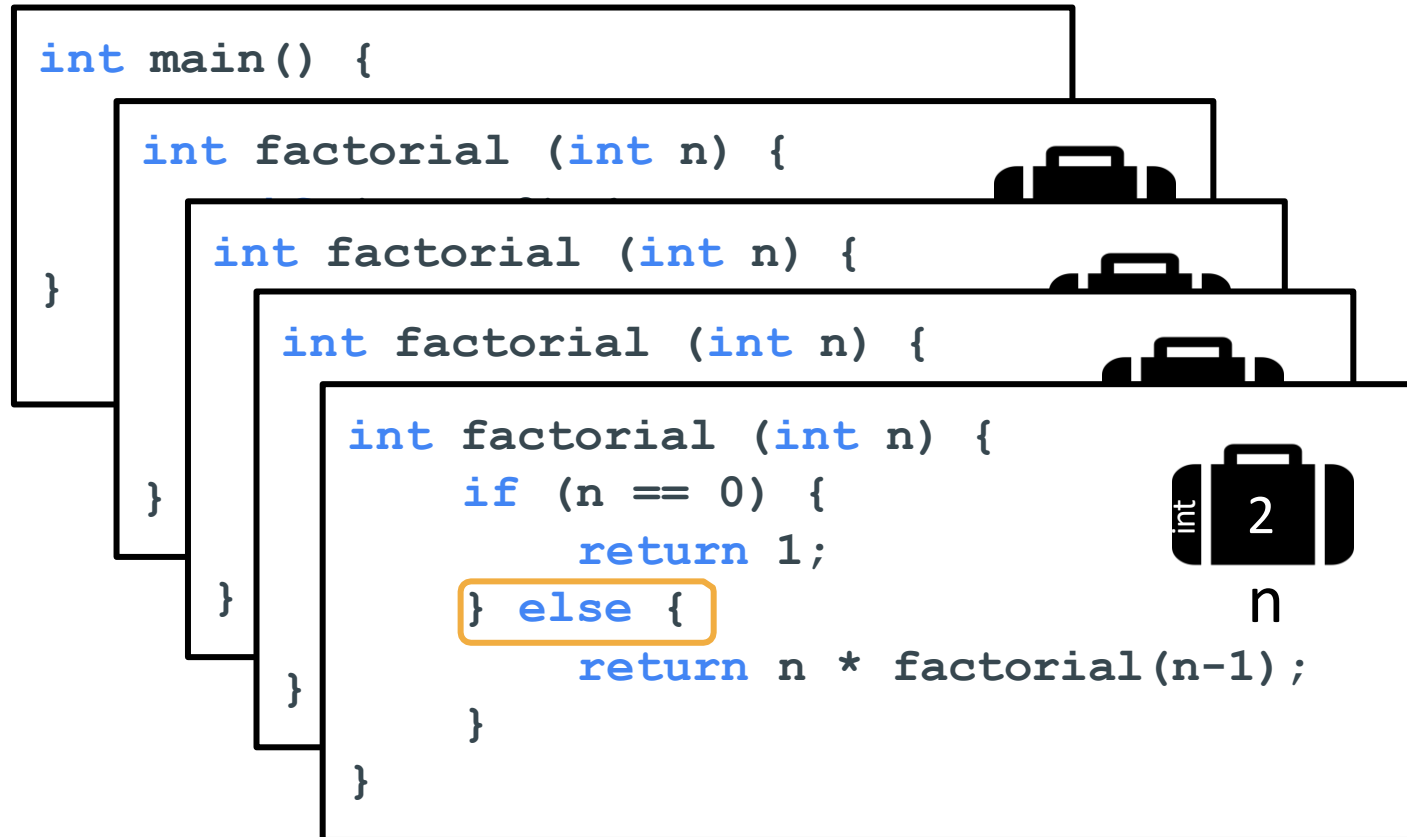
Recursion in action



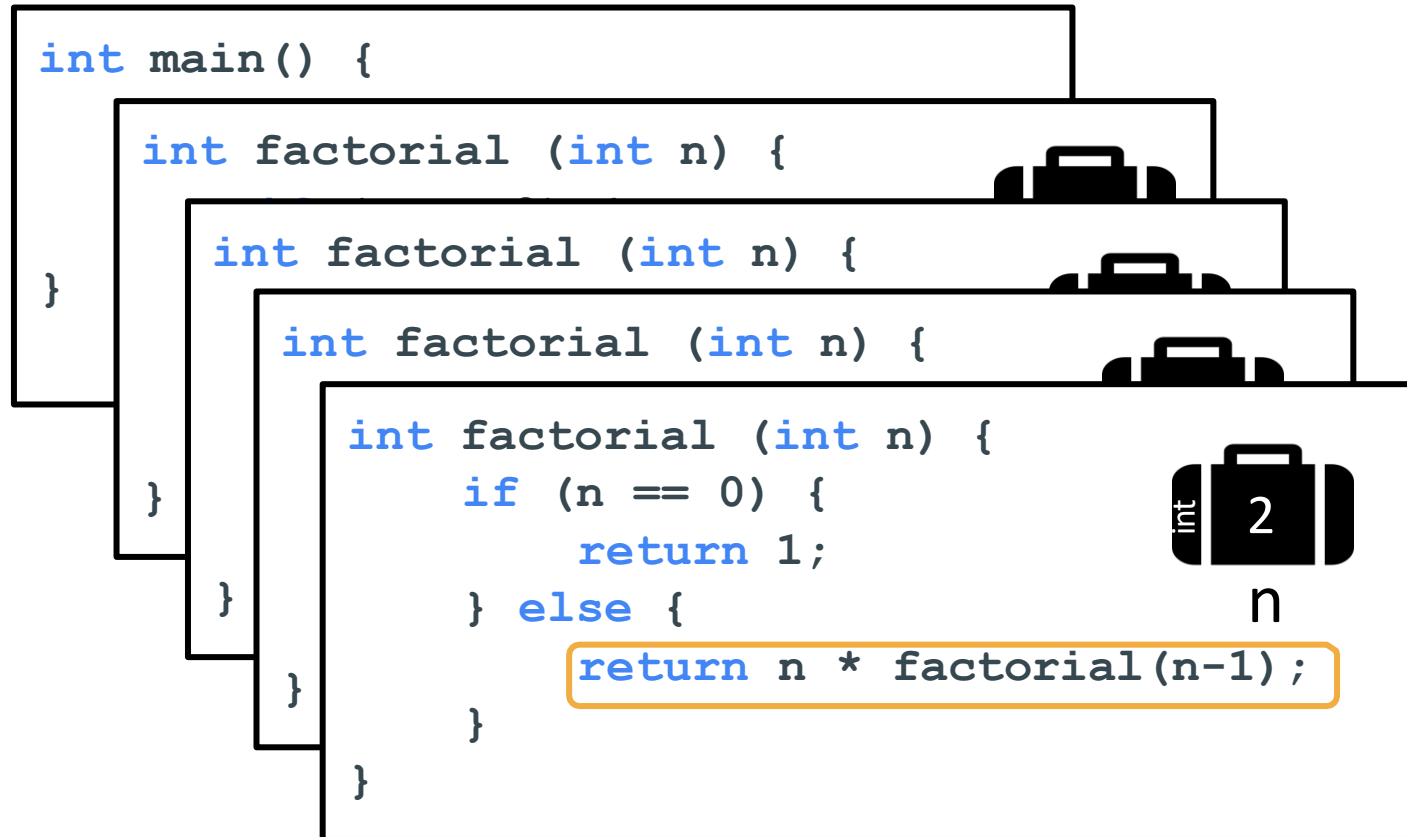
Recursion in action



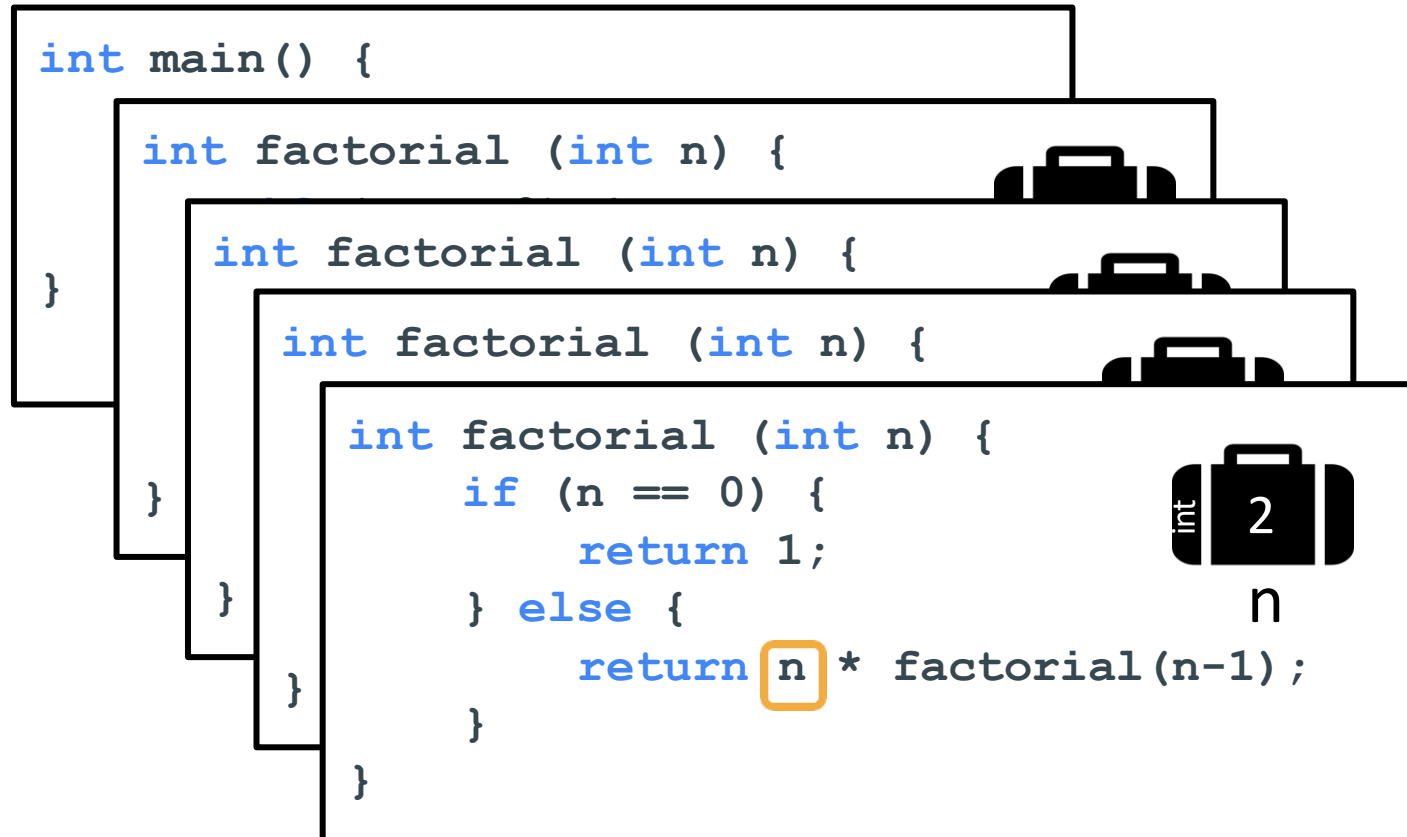
Recursion in action



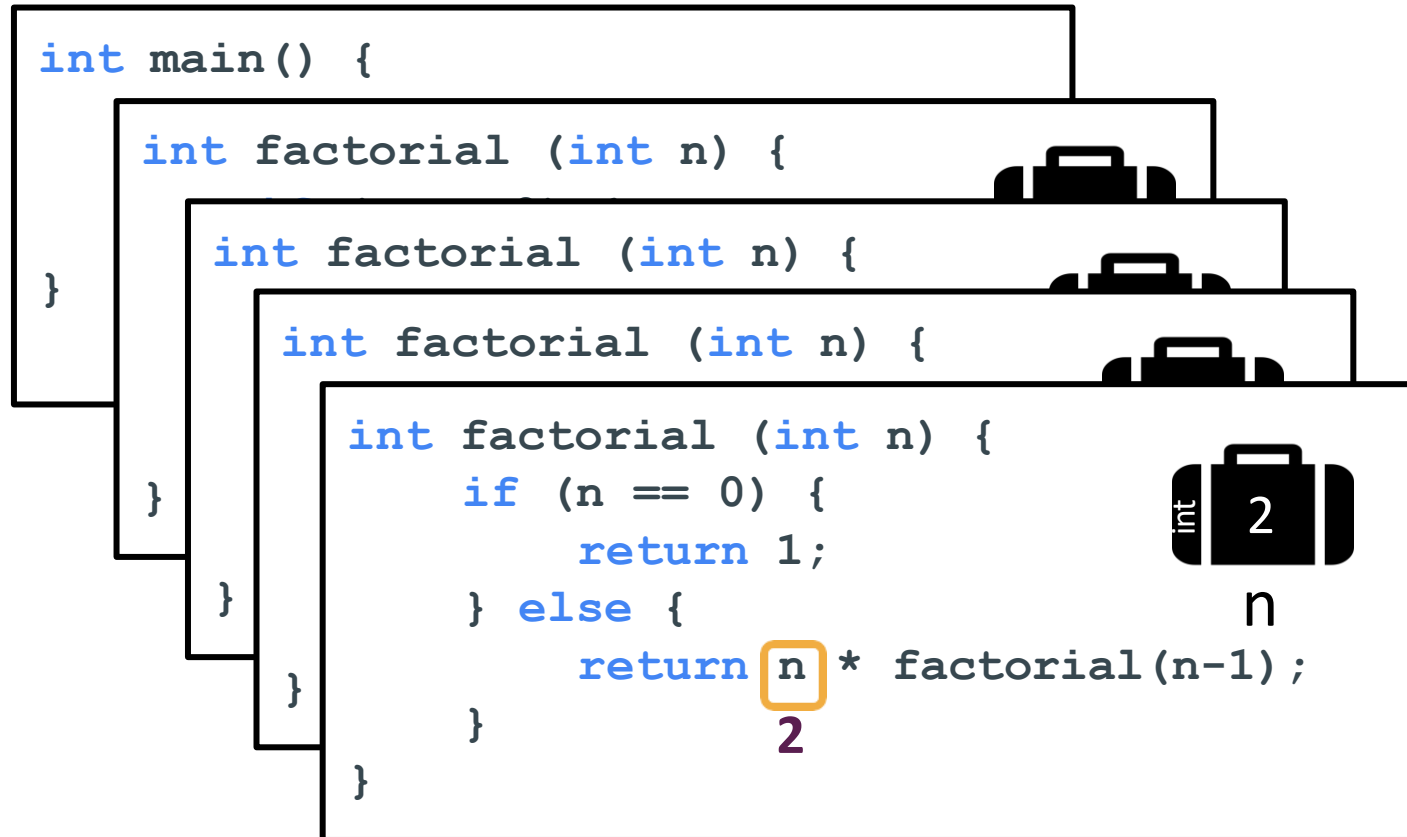
Recursion in action



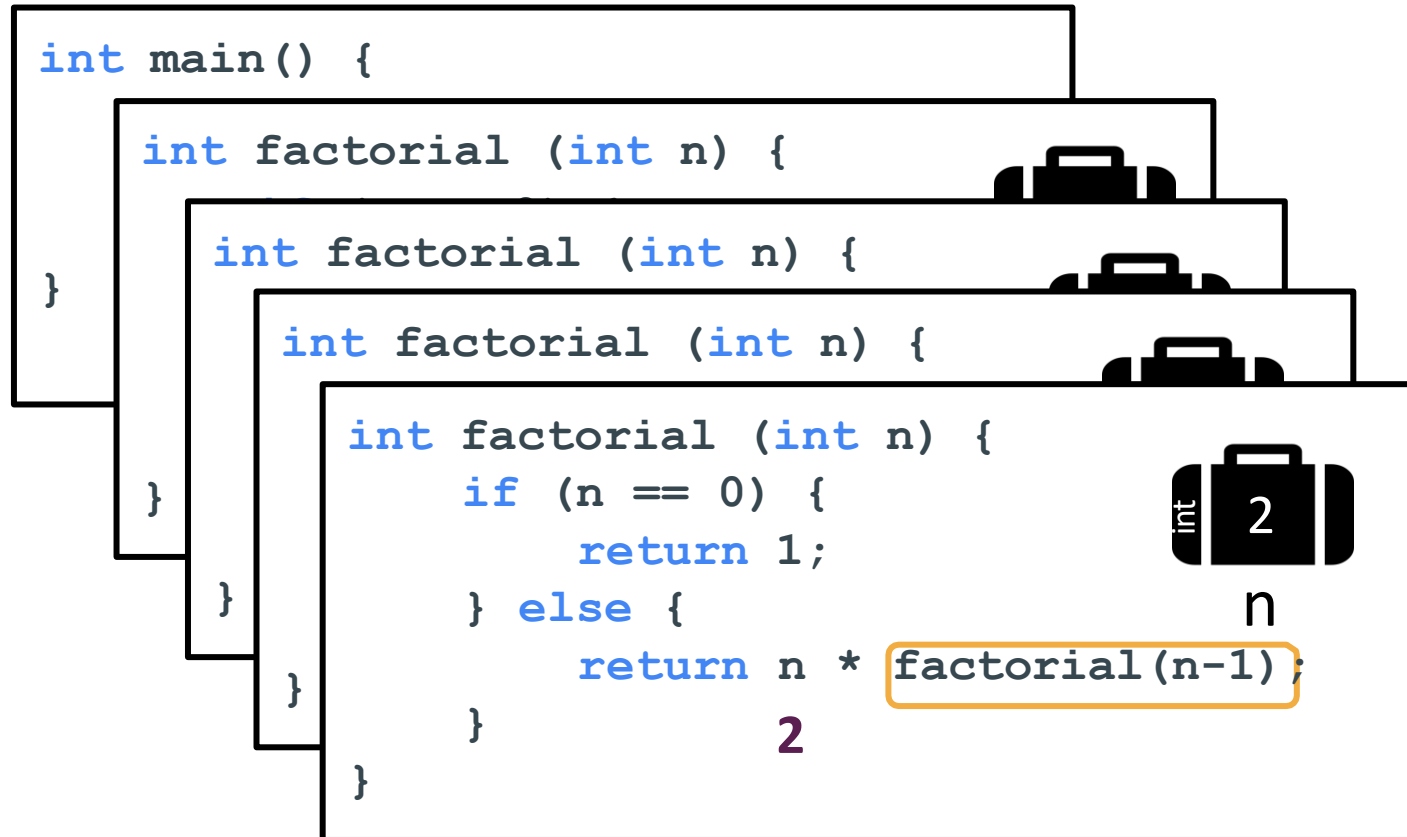
Recursion in action



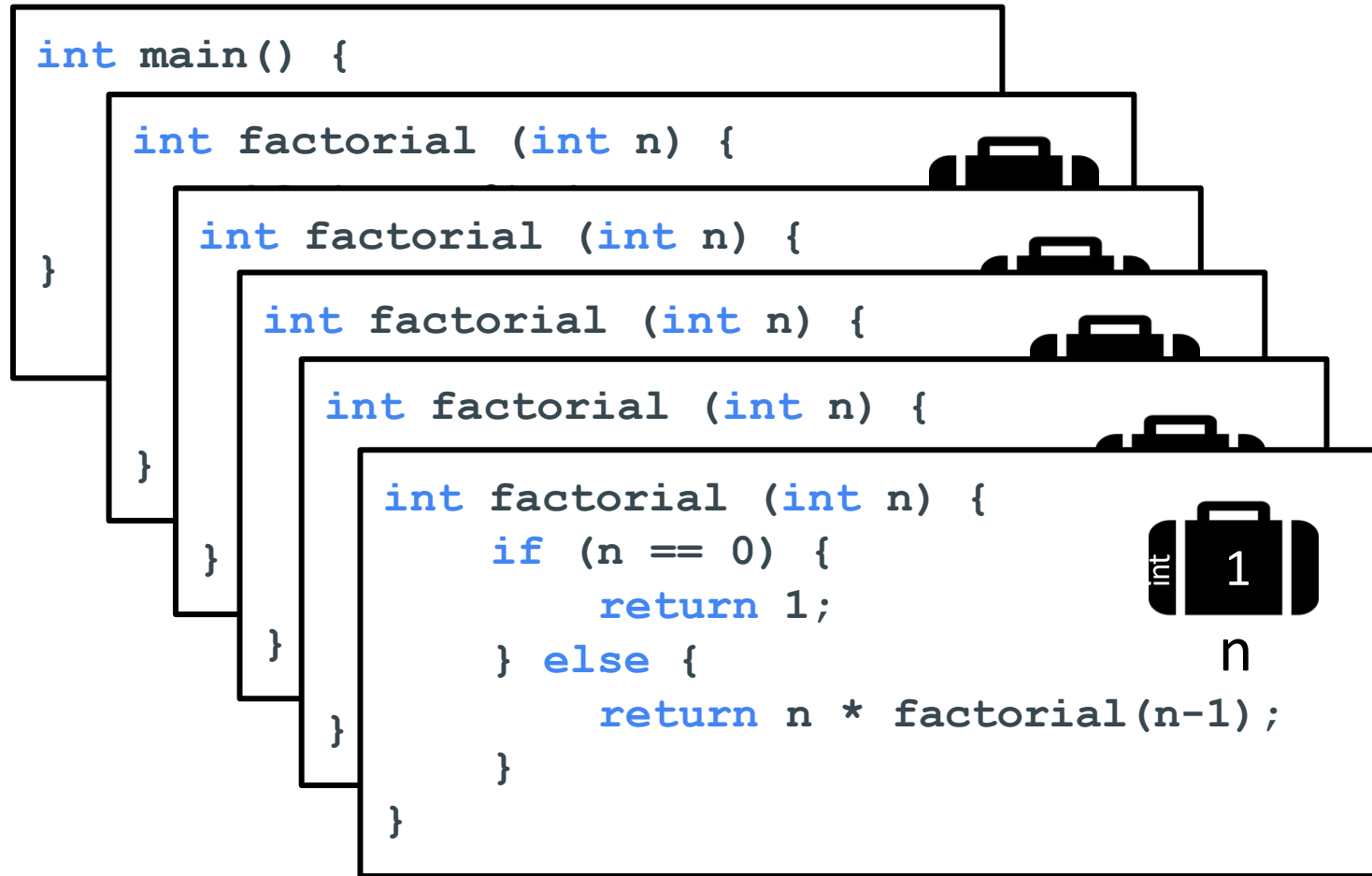
Recursion in action



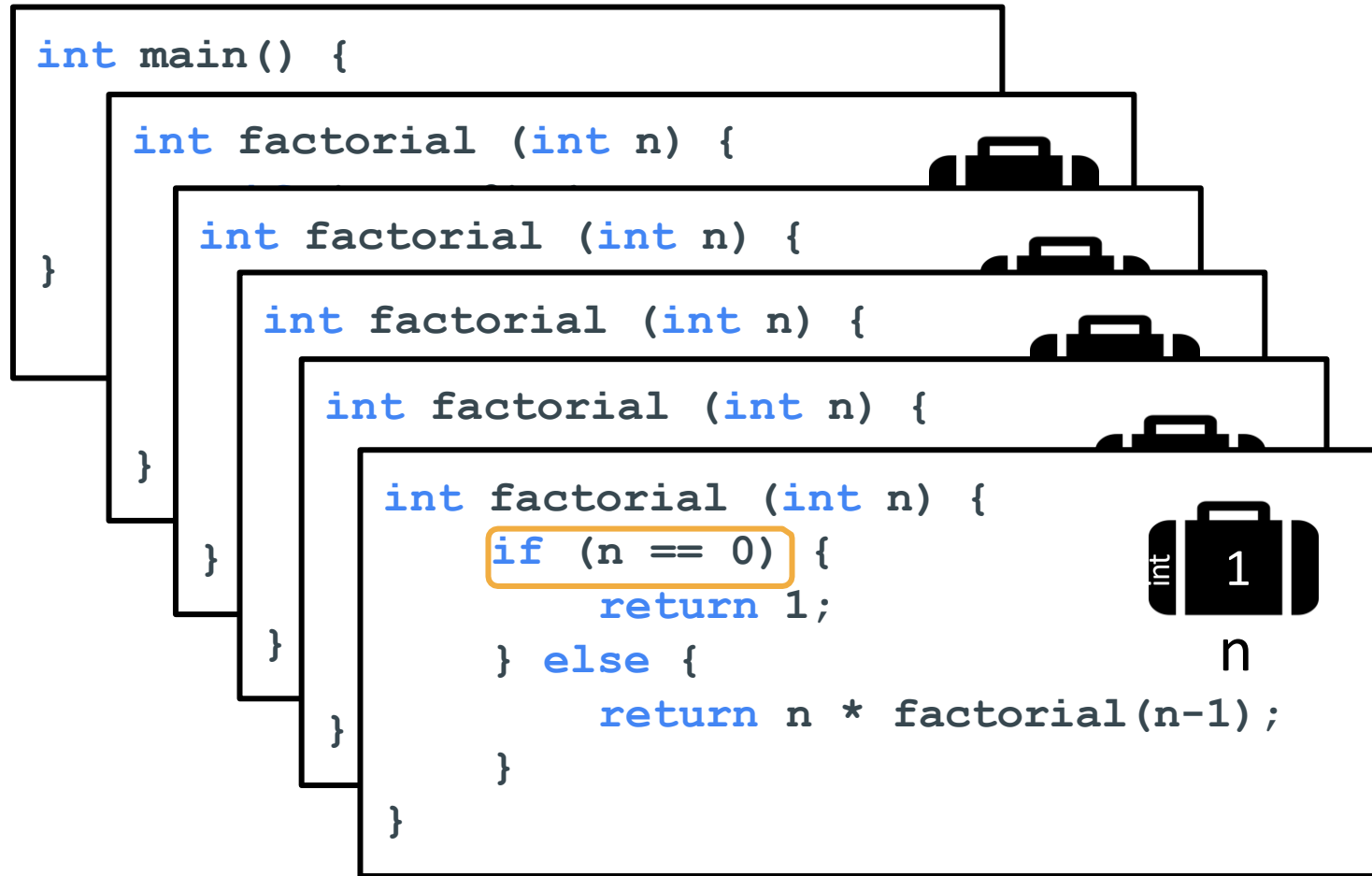
Recursion in action



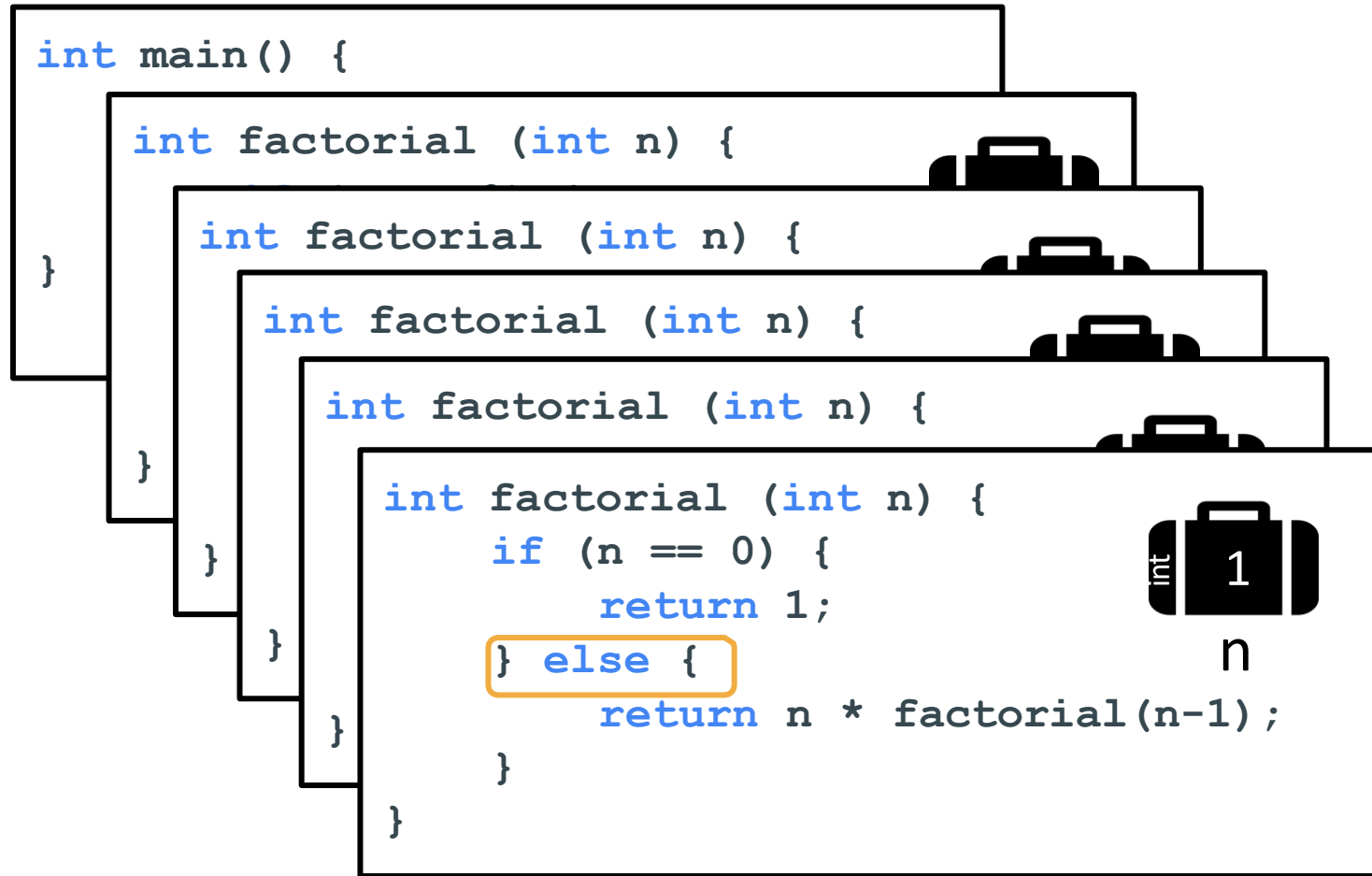
Recursion in action



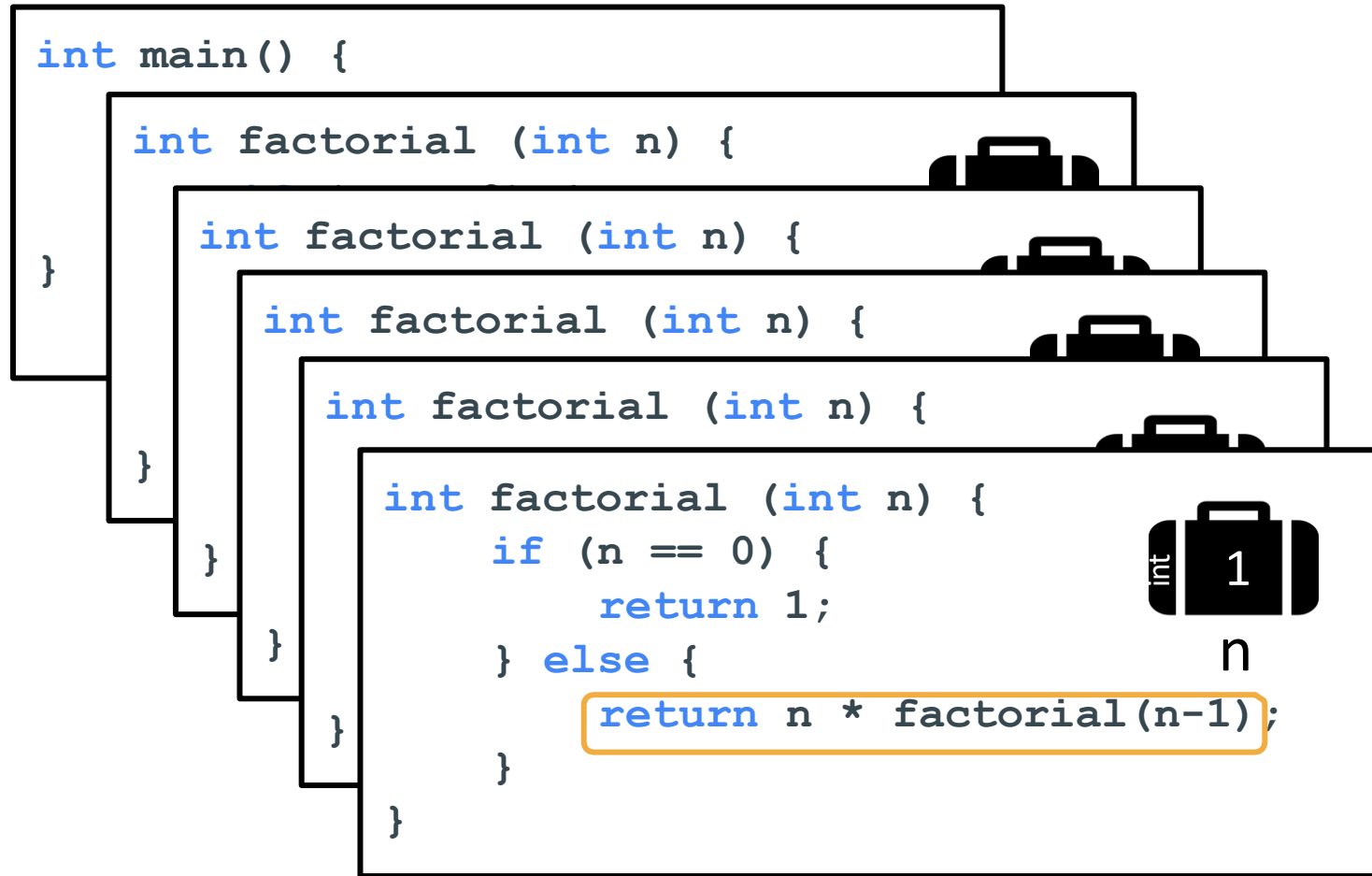
Recursion in action



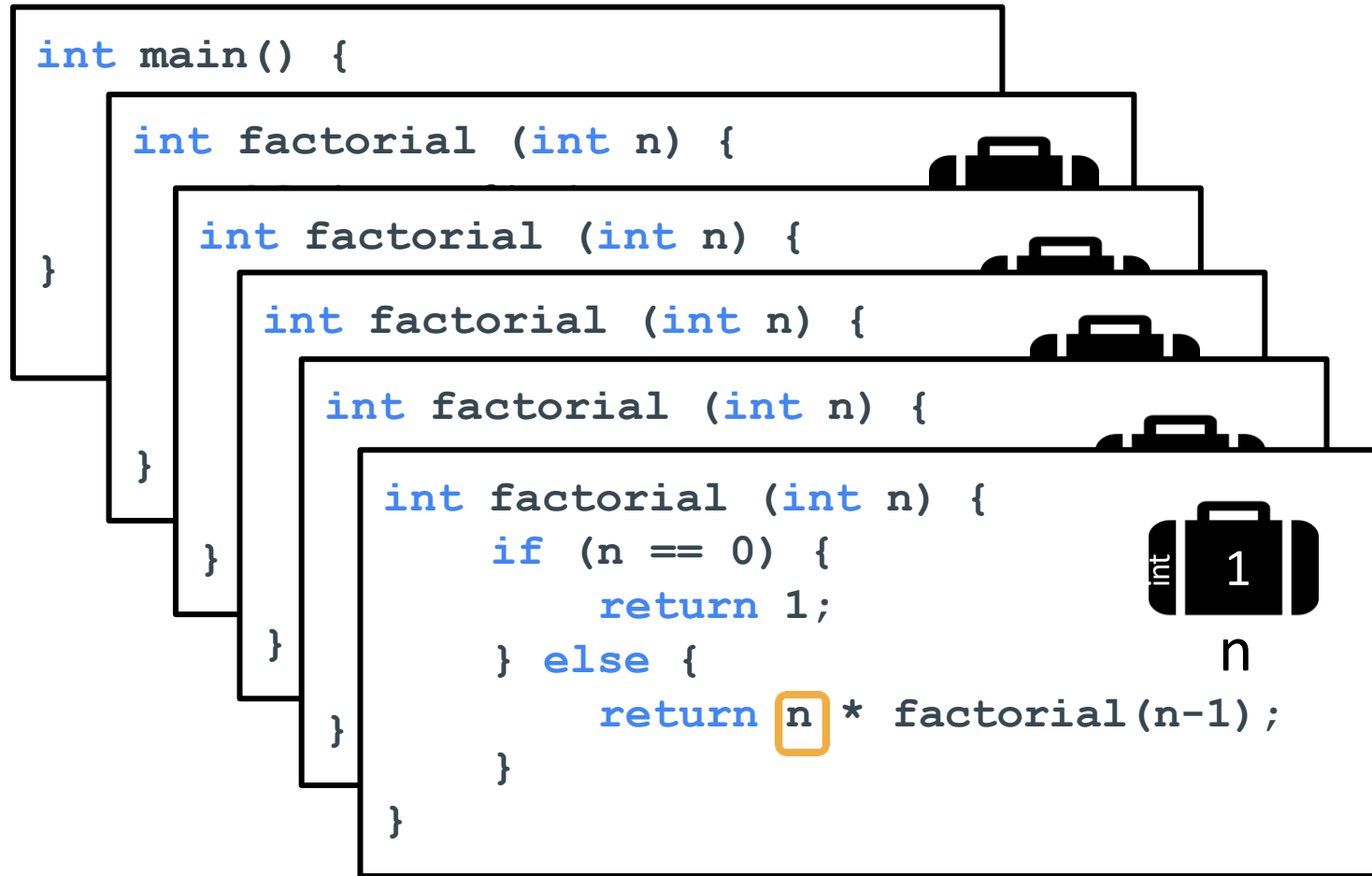
Recursion in action



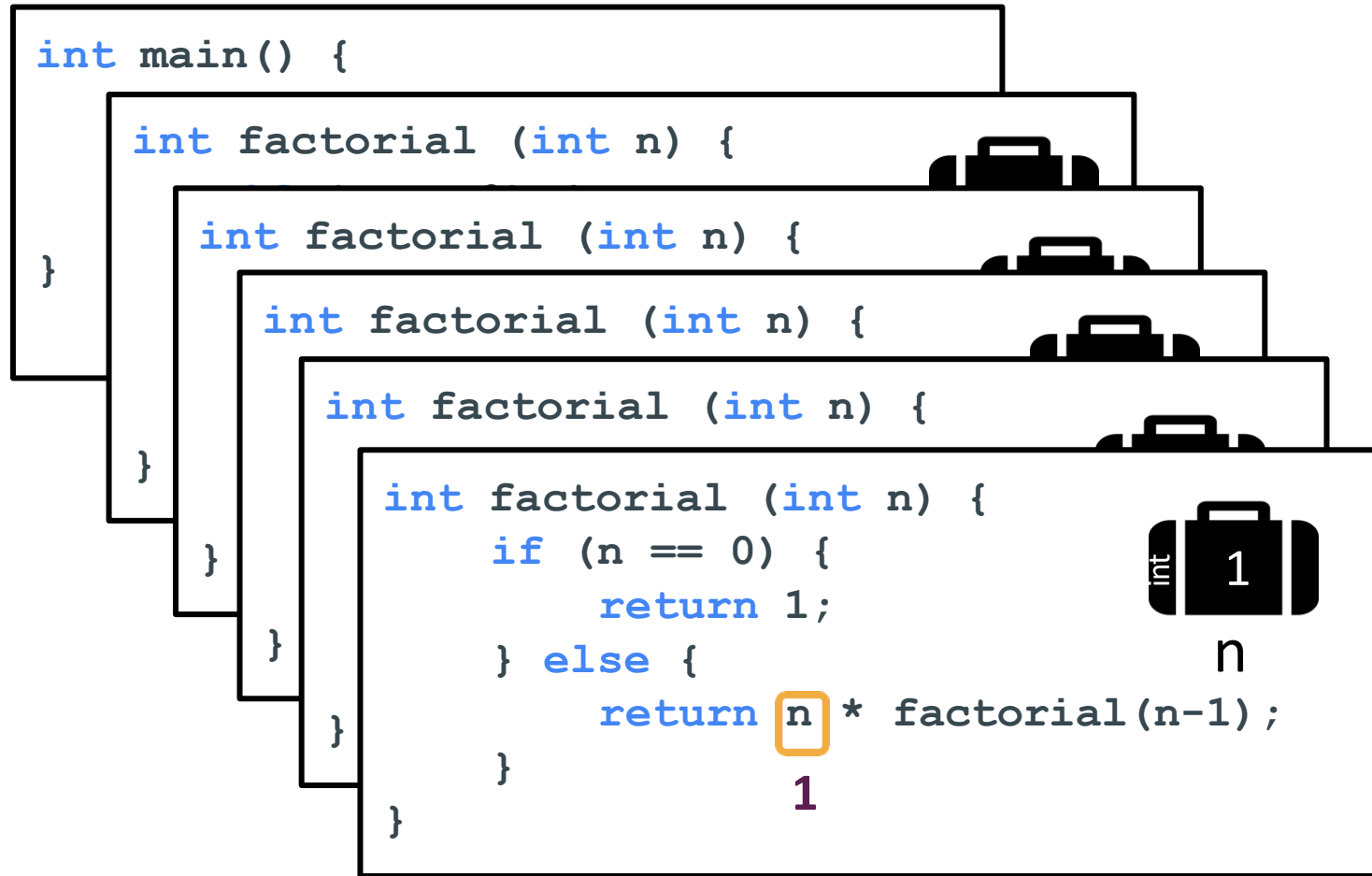
Recursion in action



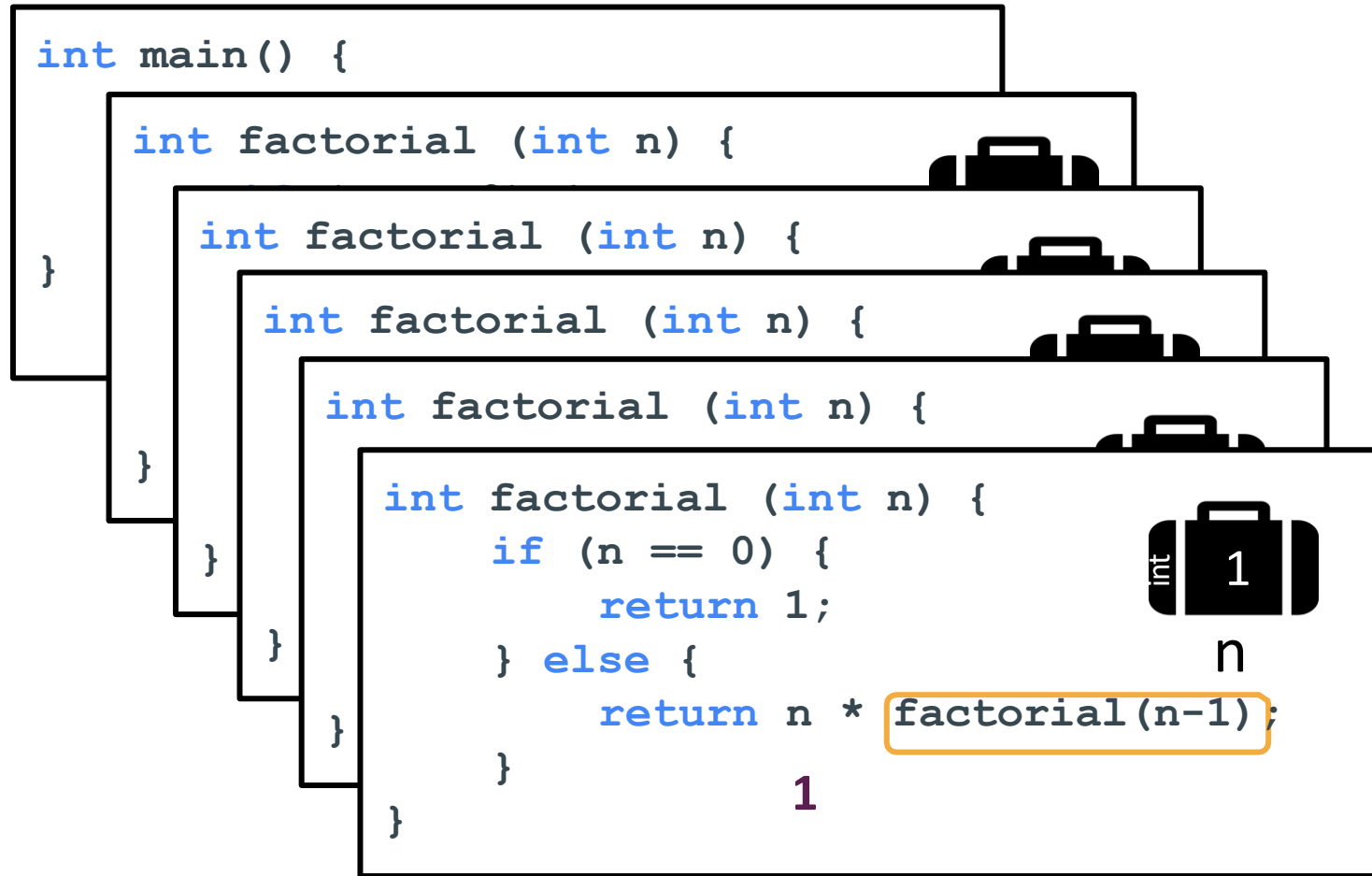
Recursion in action



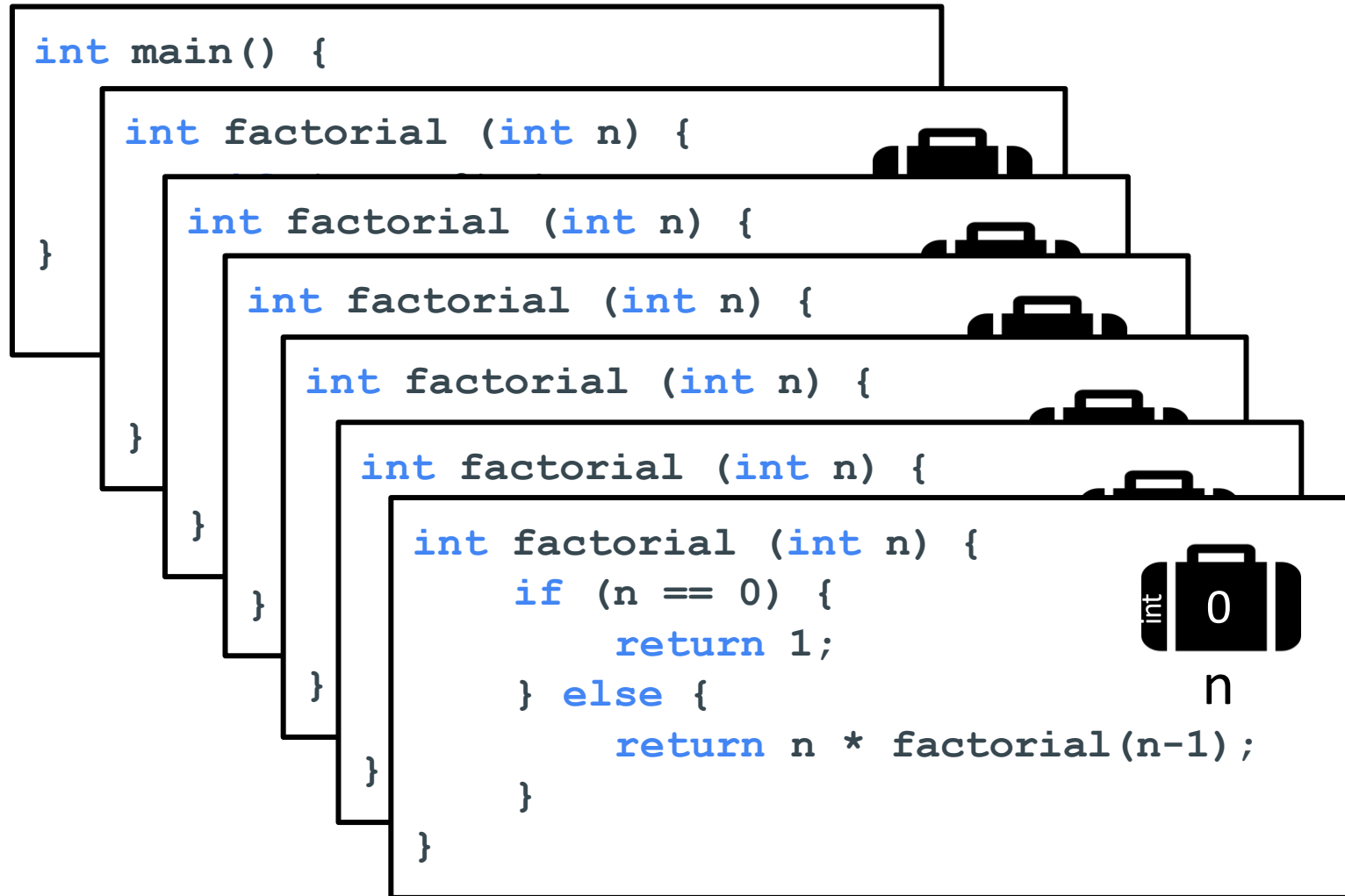
Recursion in action



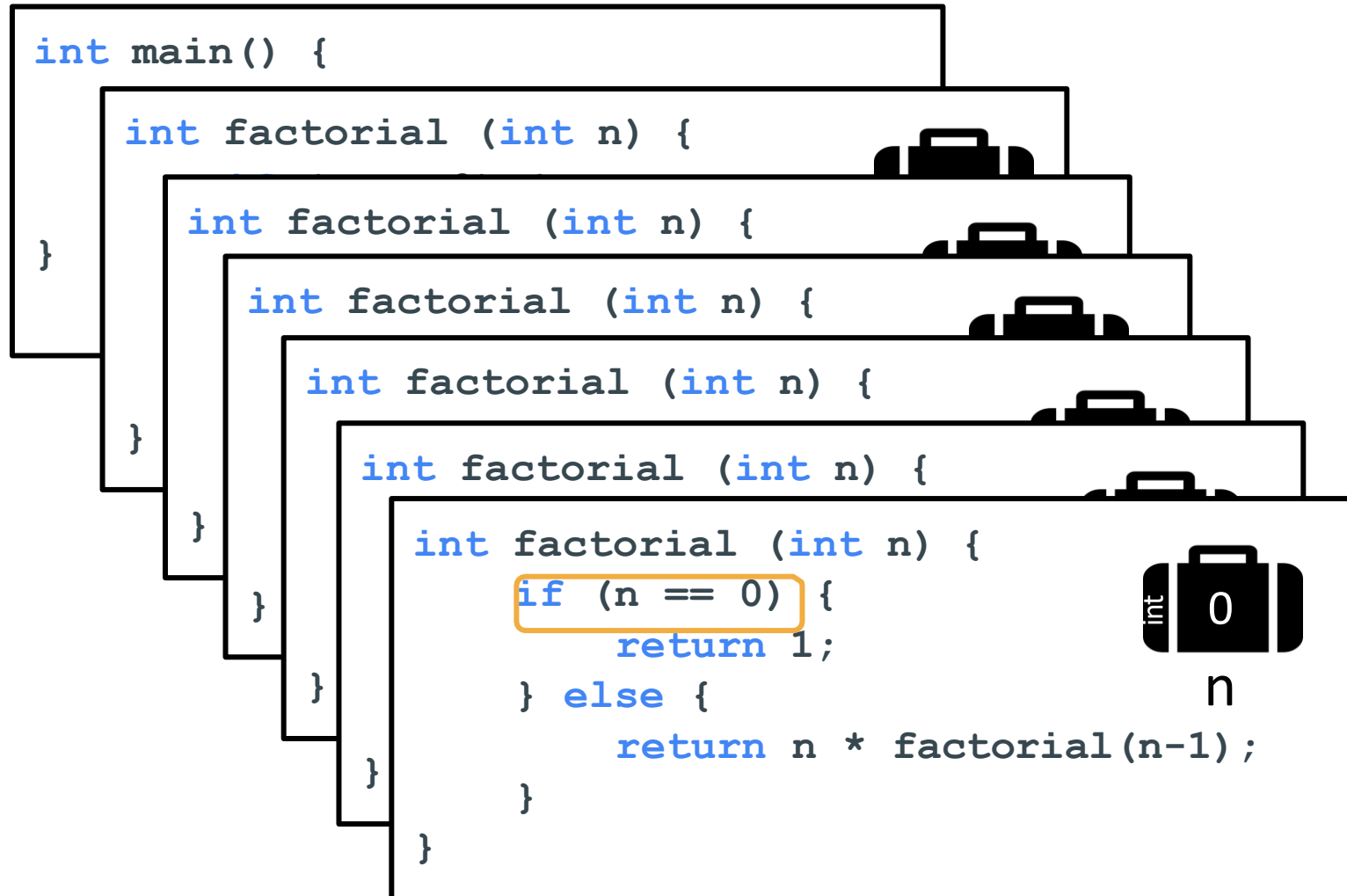
Recursion in action



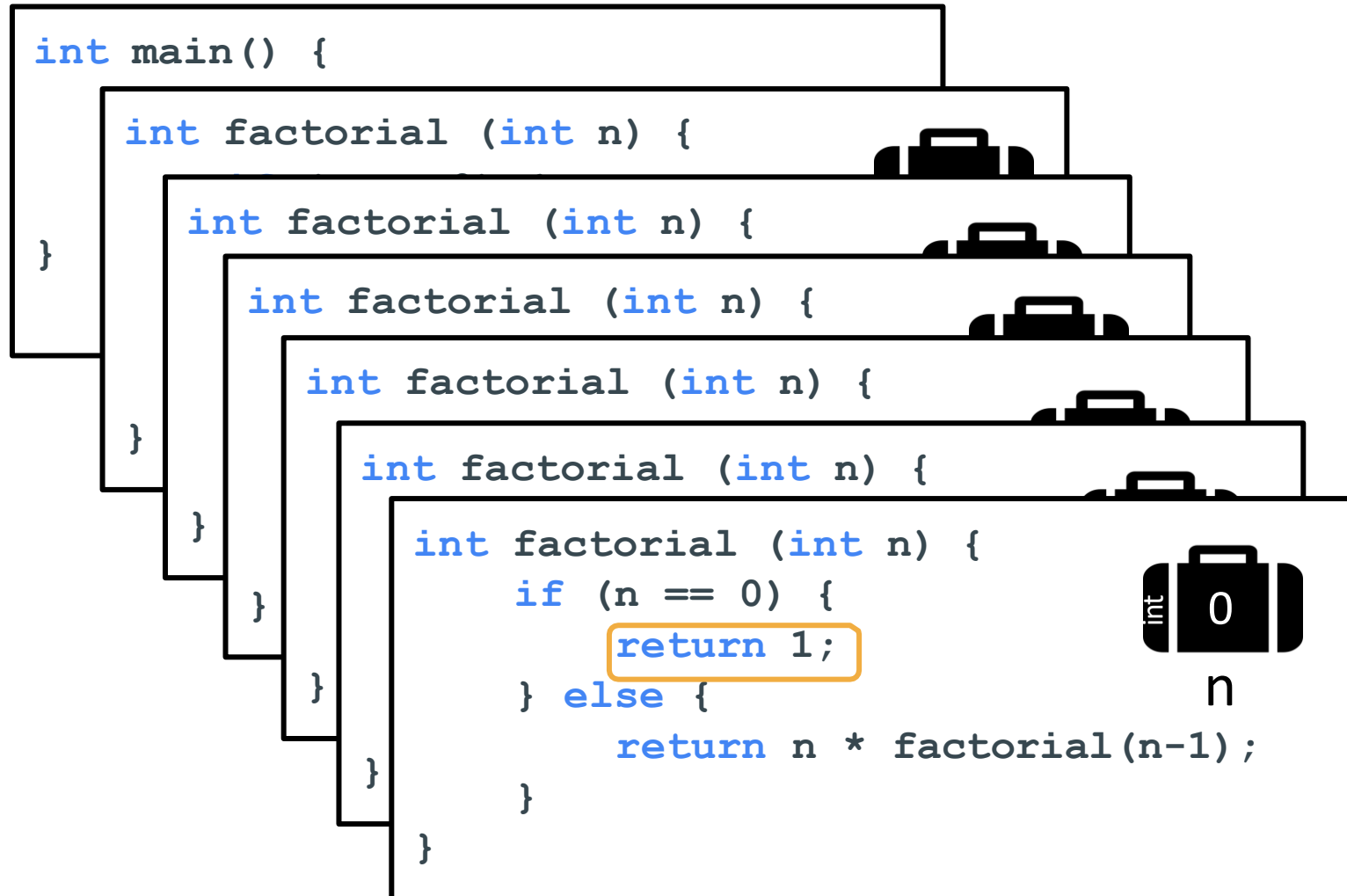
Recursion in action



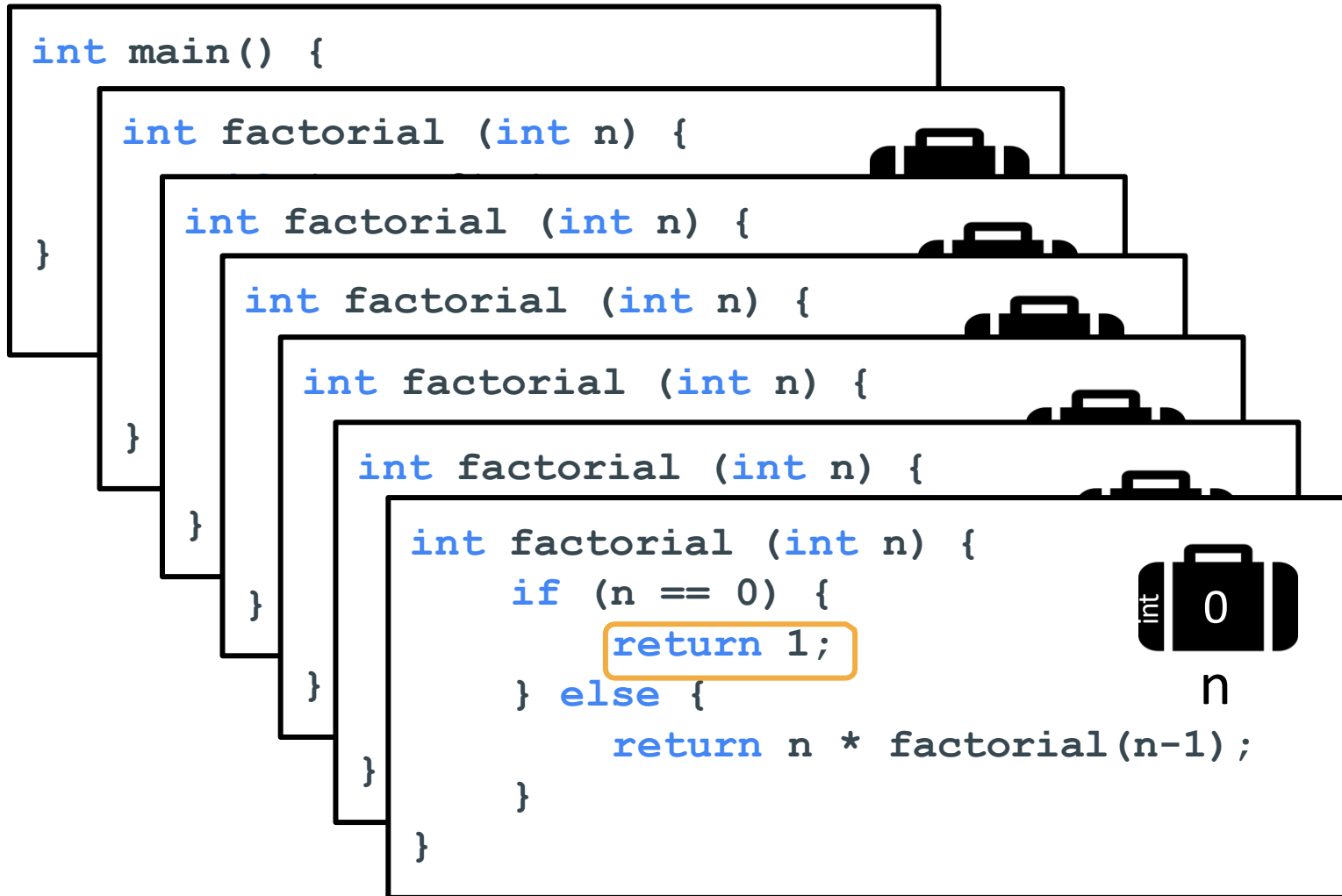
Recursion in action



Recursion in action



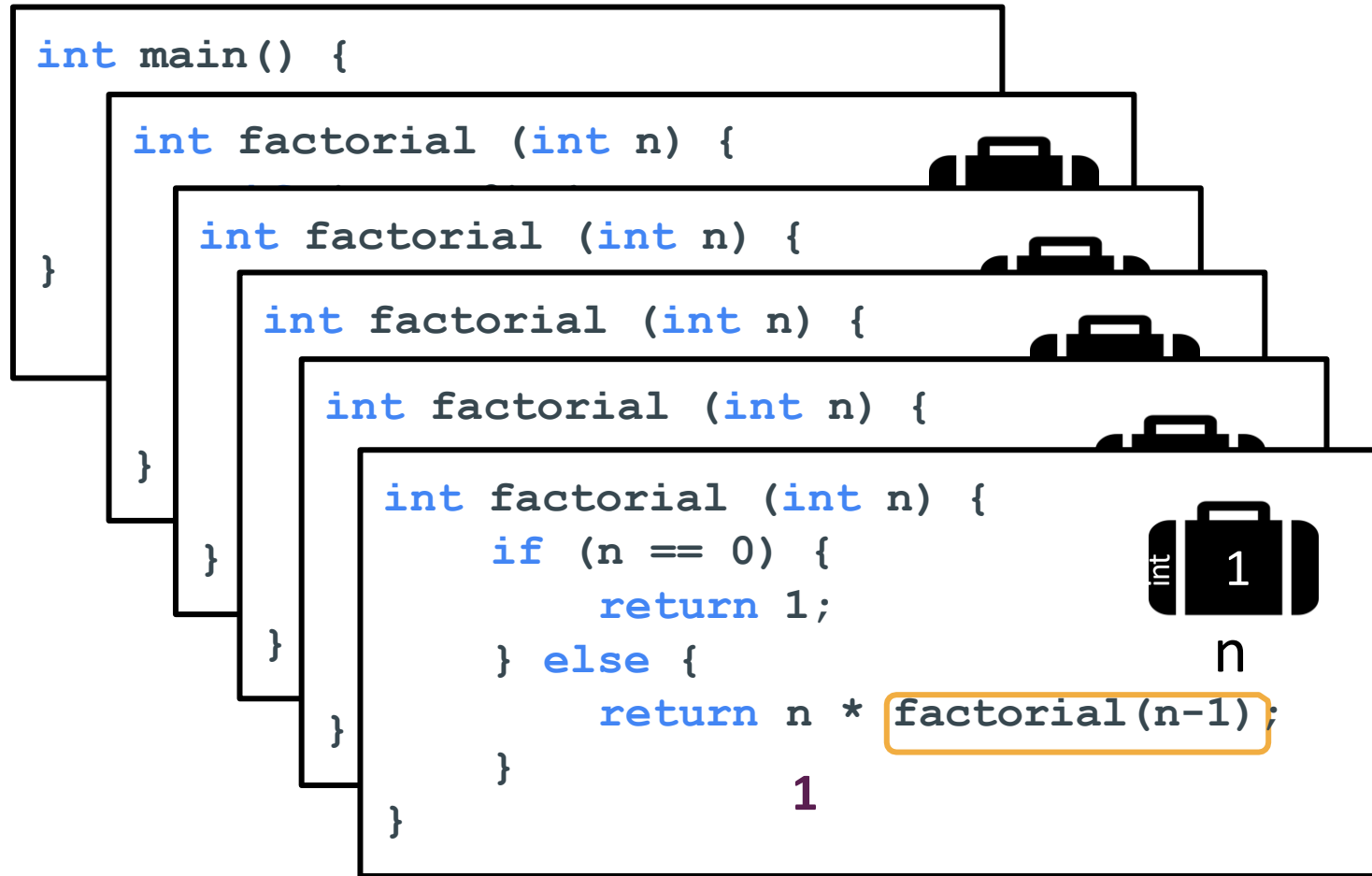
Recursion in action



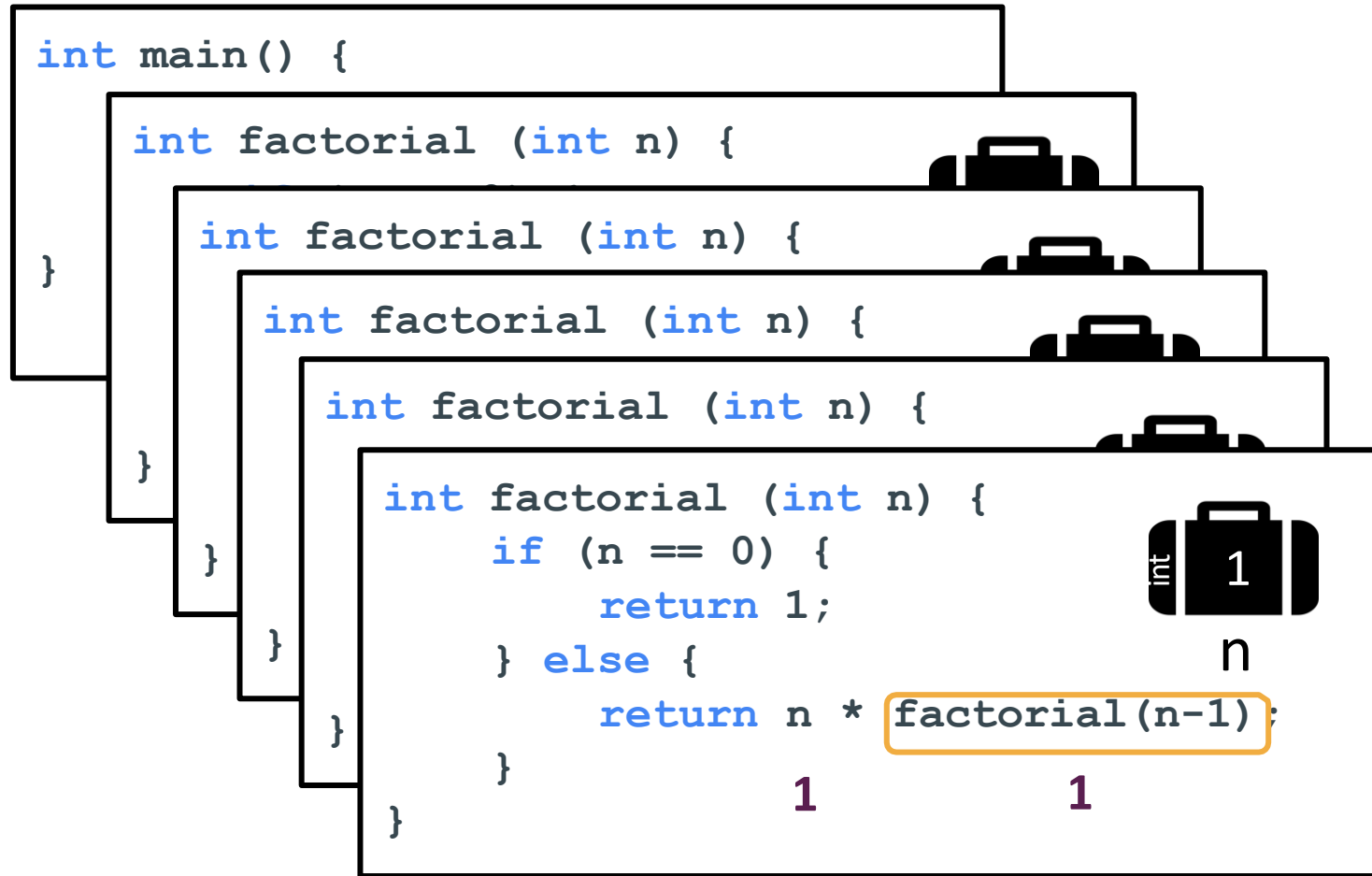
Stack frames go away (get cleared from memory) once they return.



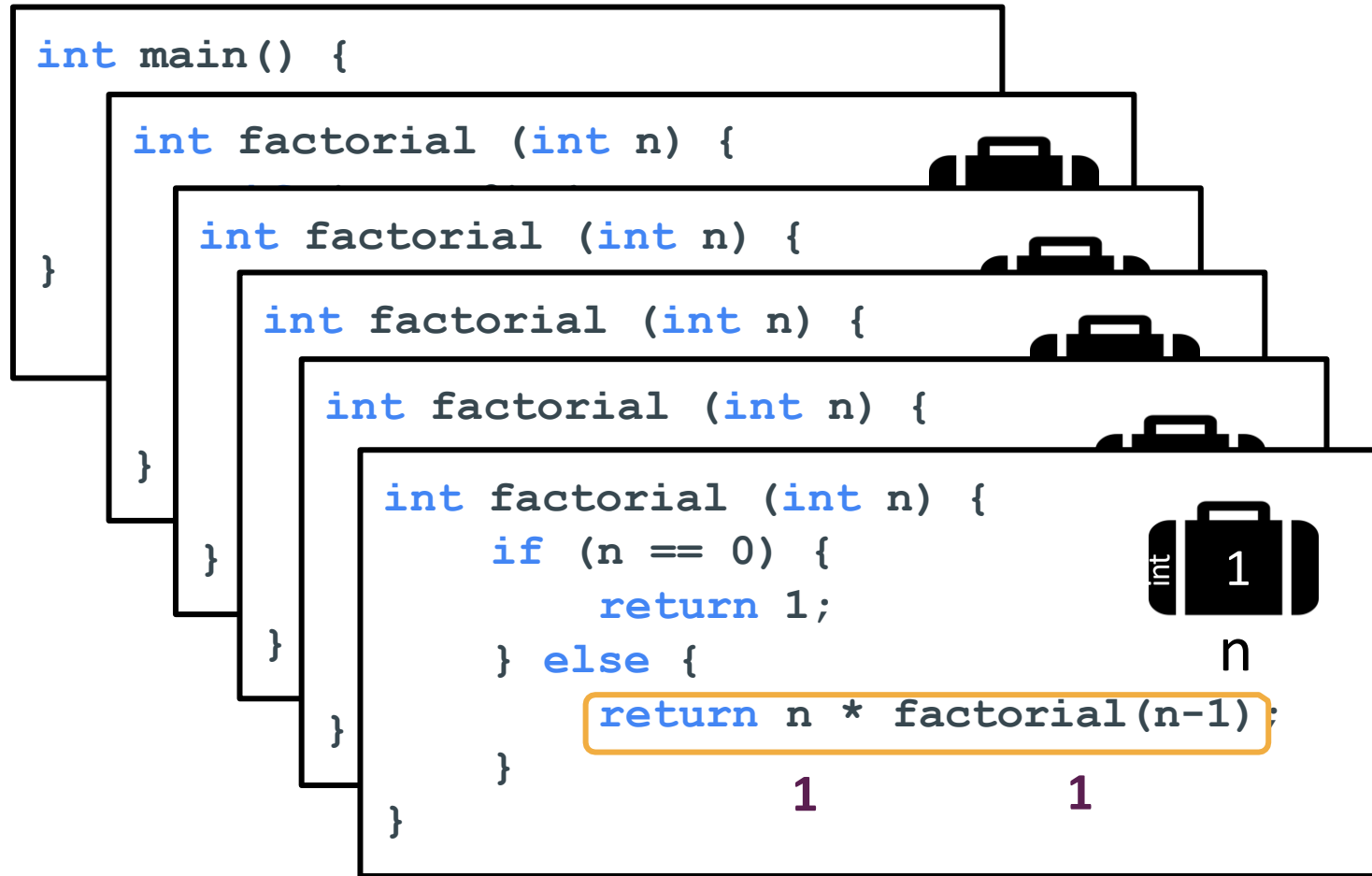
Recursion in action



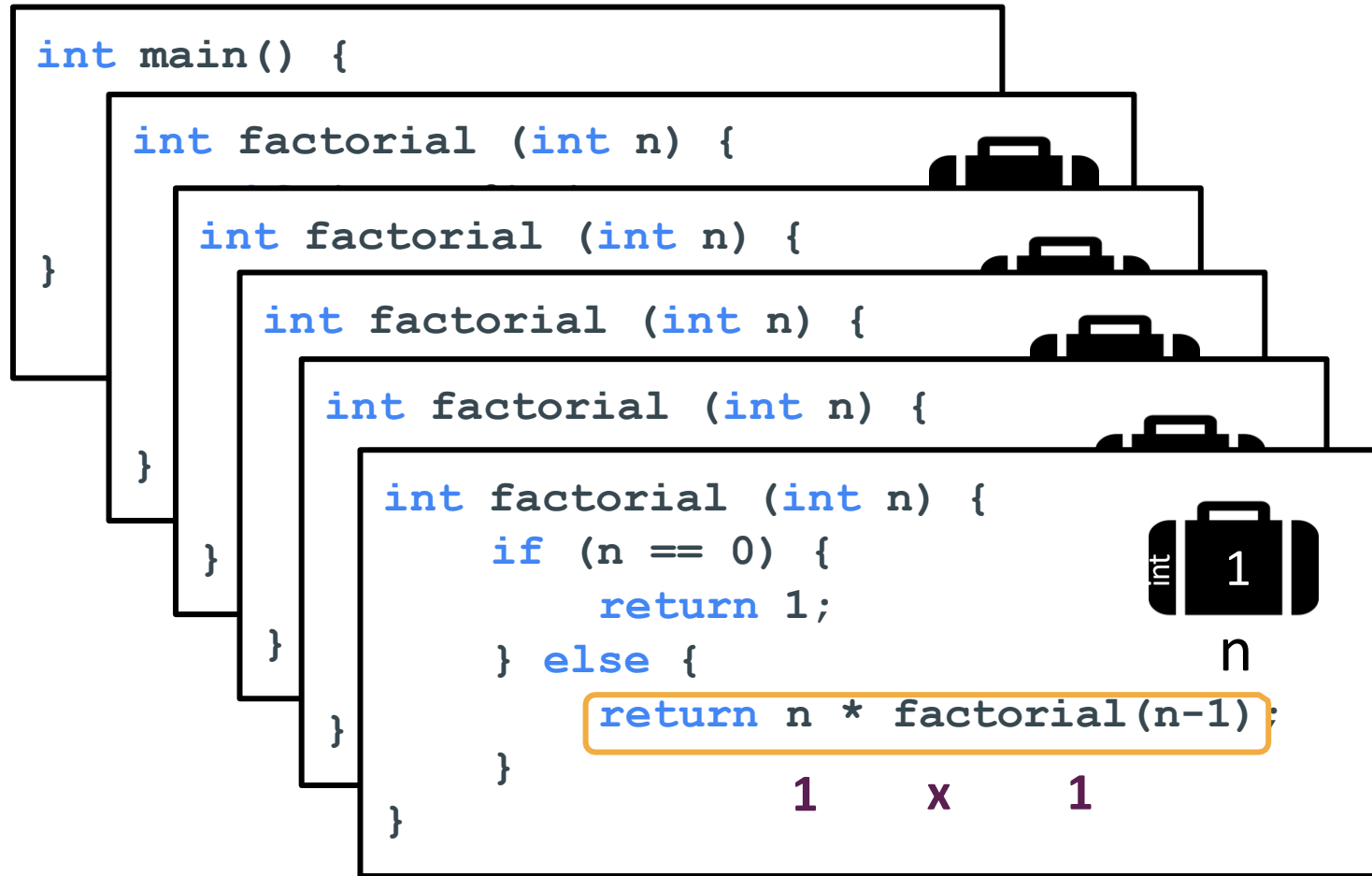
Recursion in action



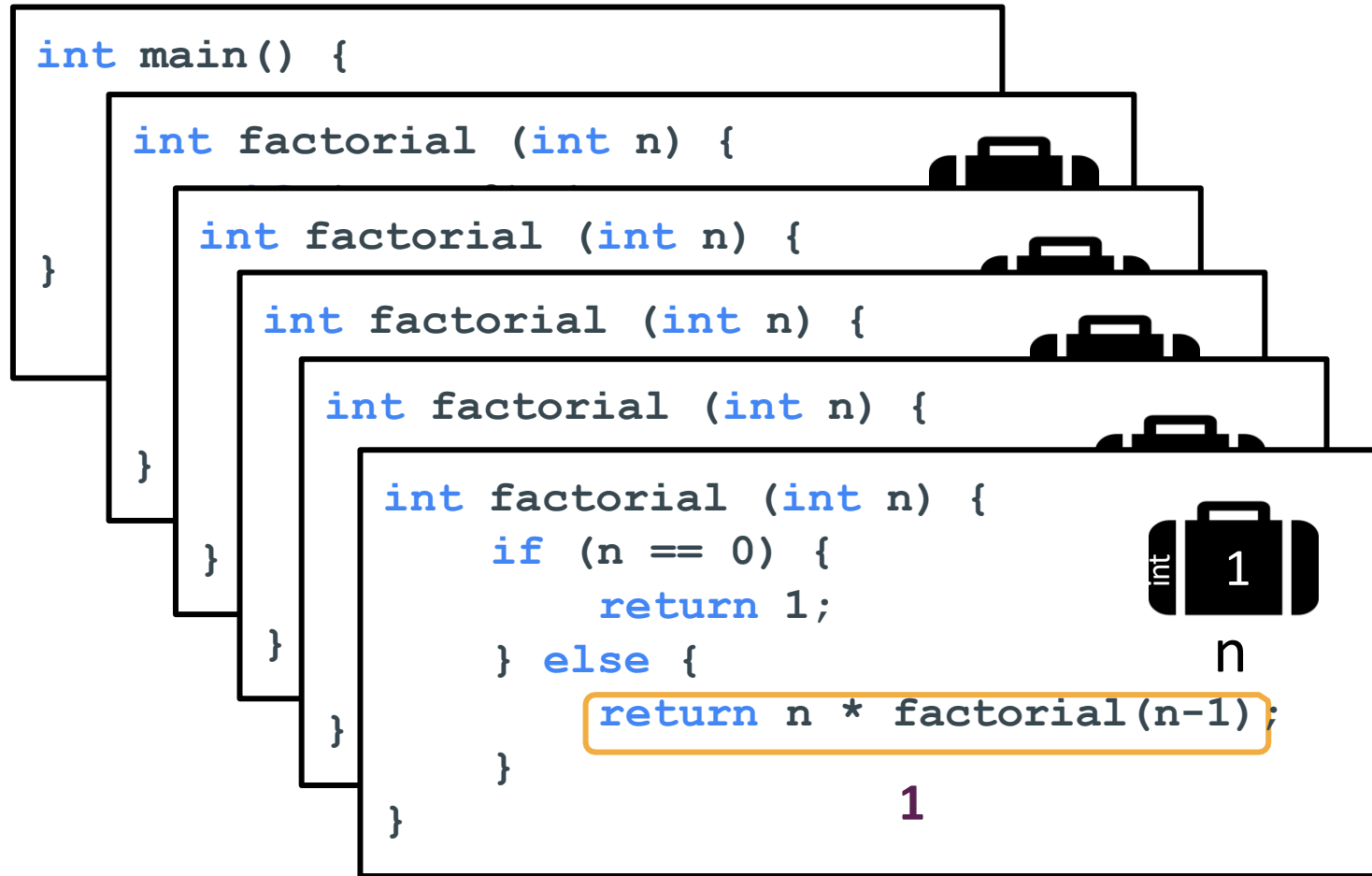
Recursion in action



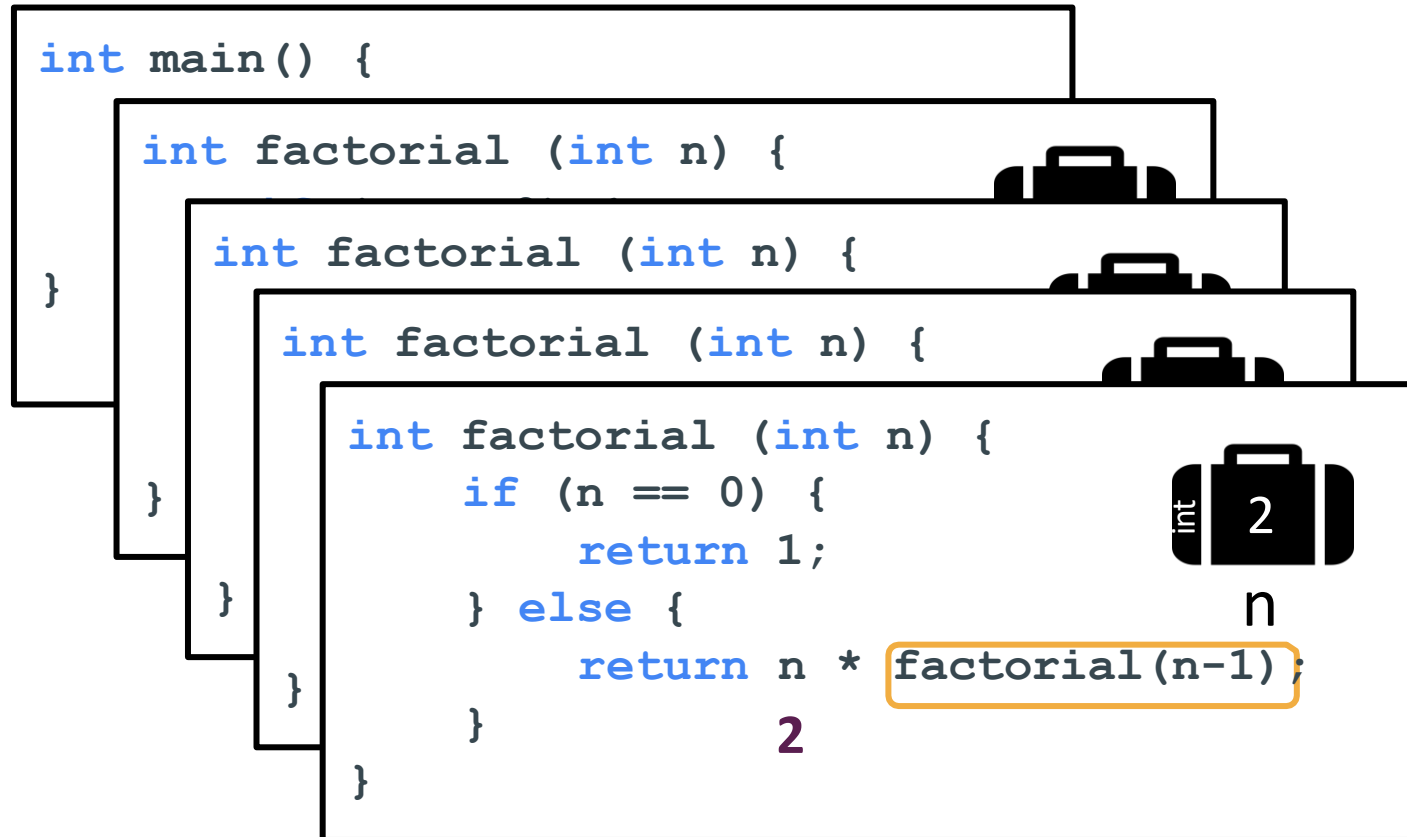
Recursion in action



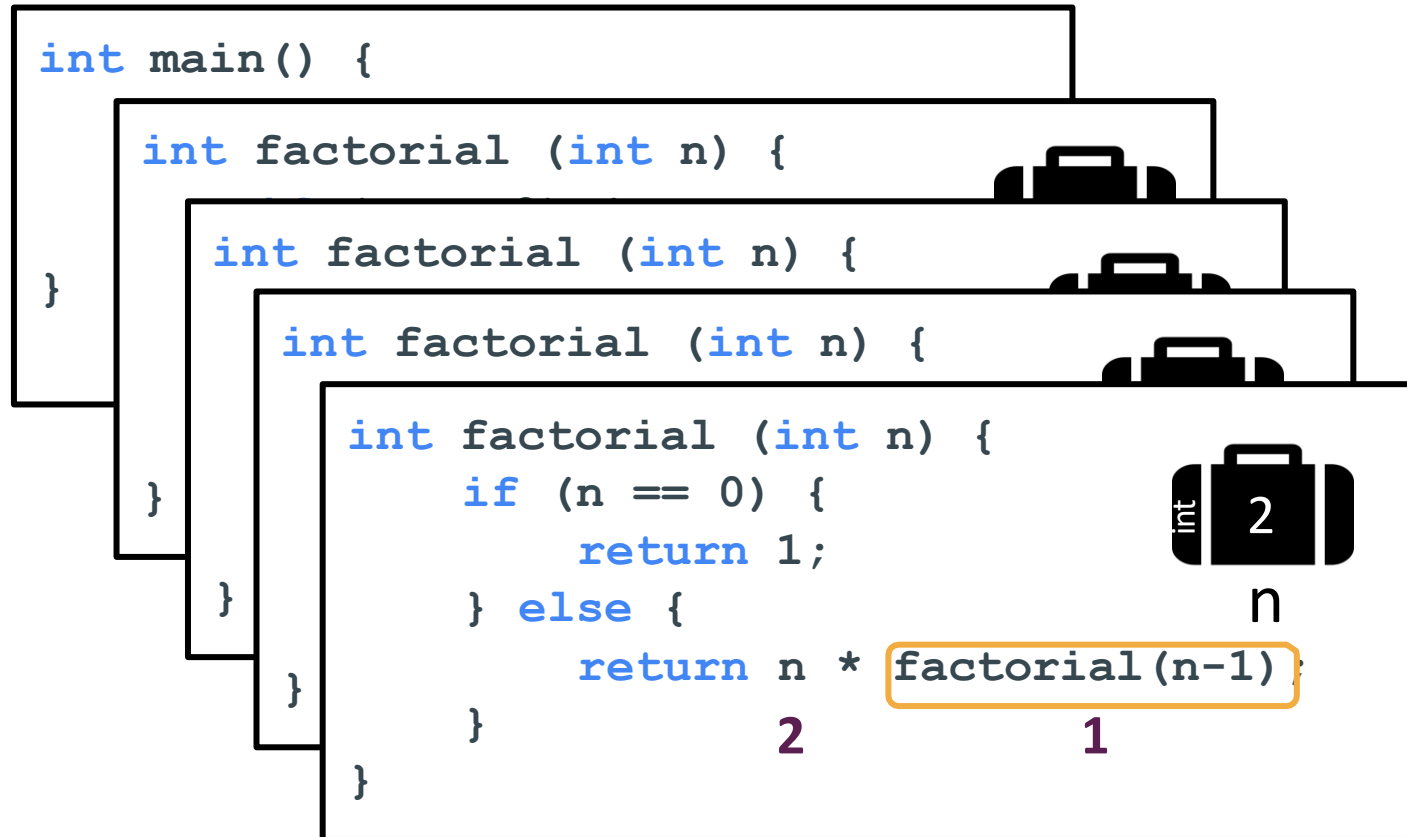
Recursion in action



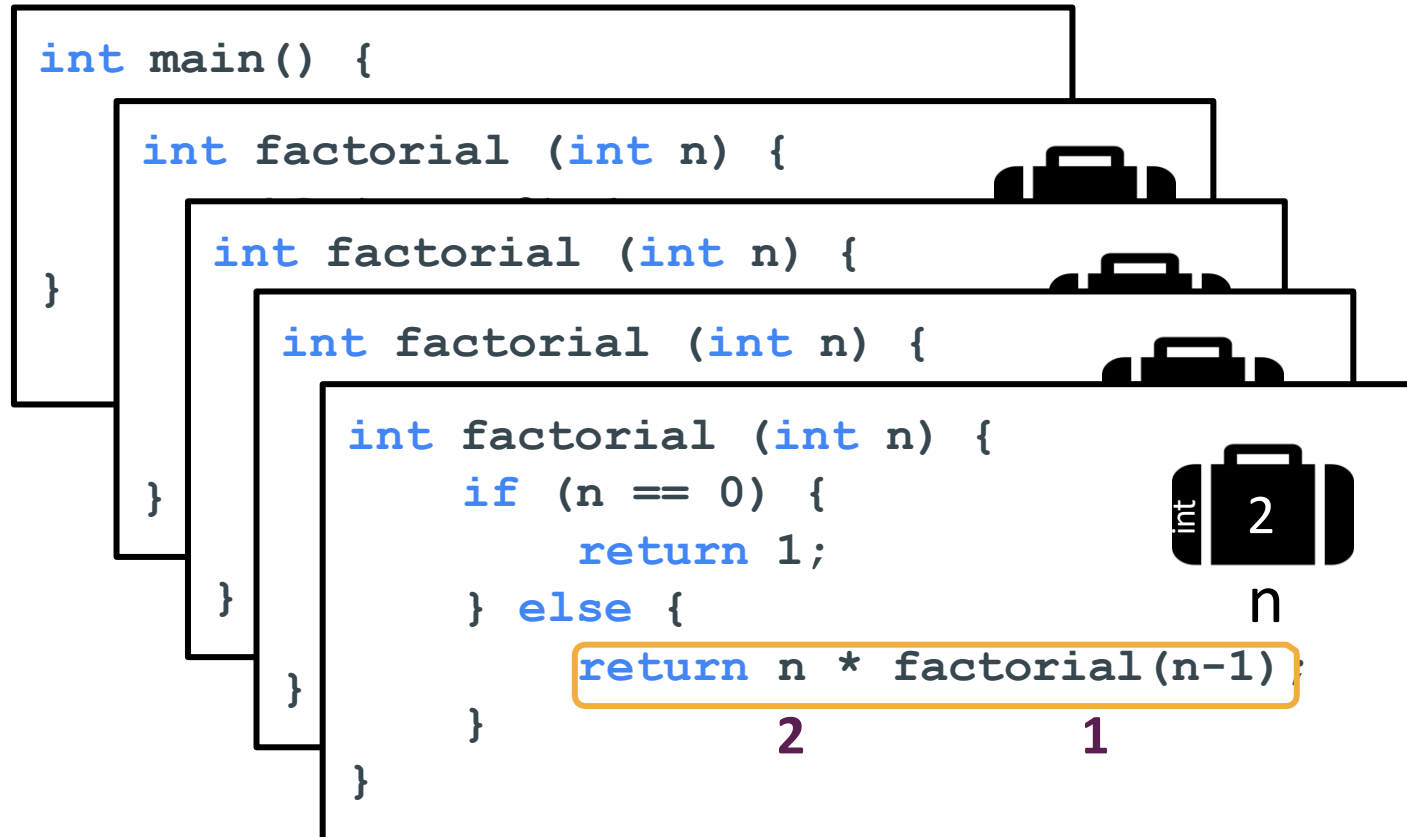
Recursion in action



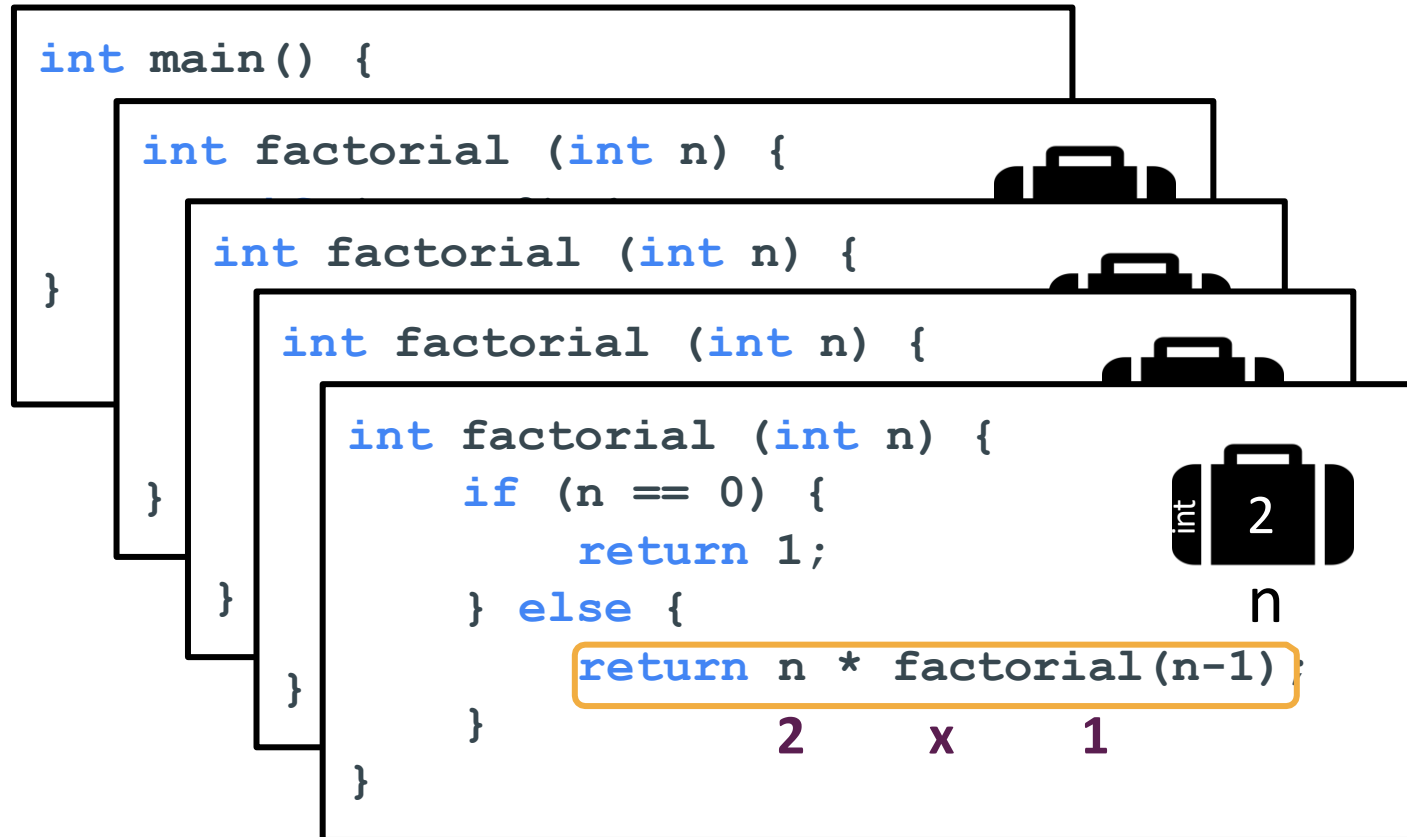
Recursion in action



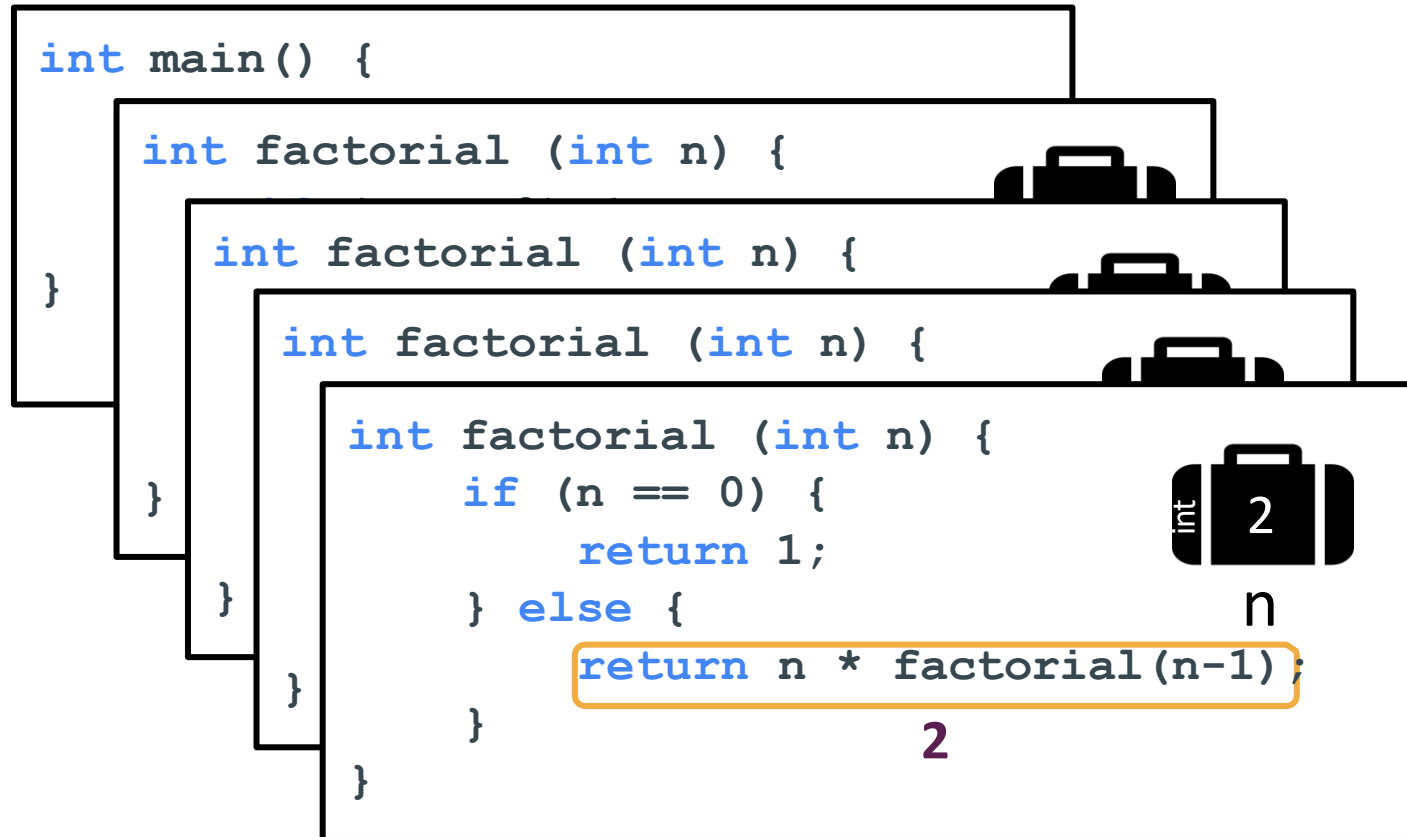
Recursion in action



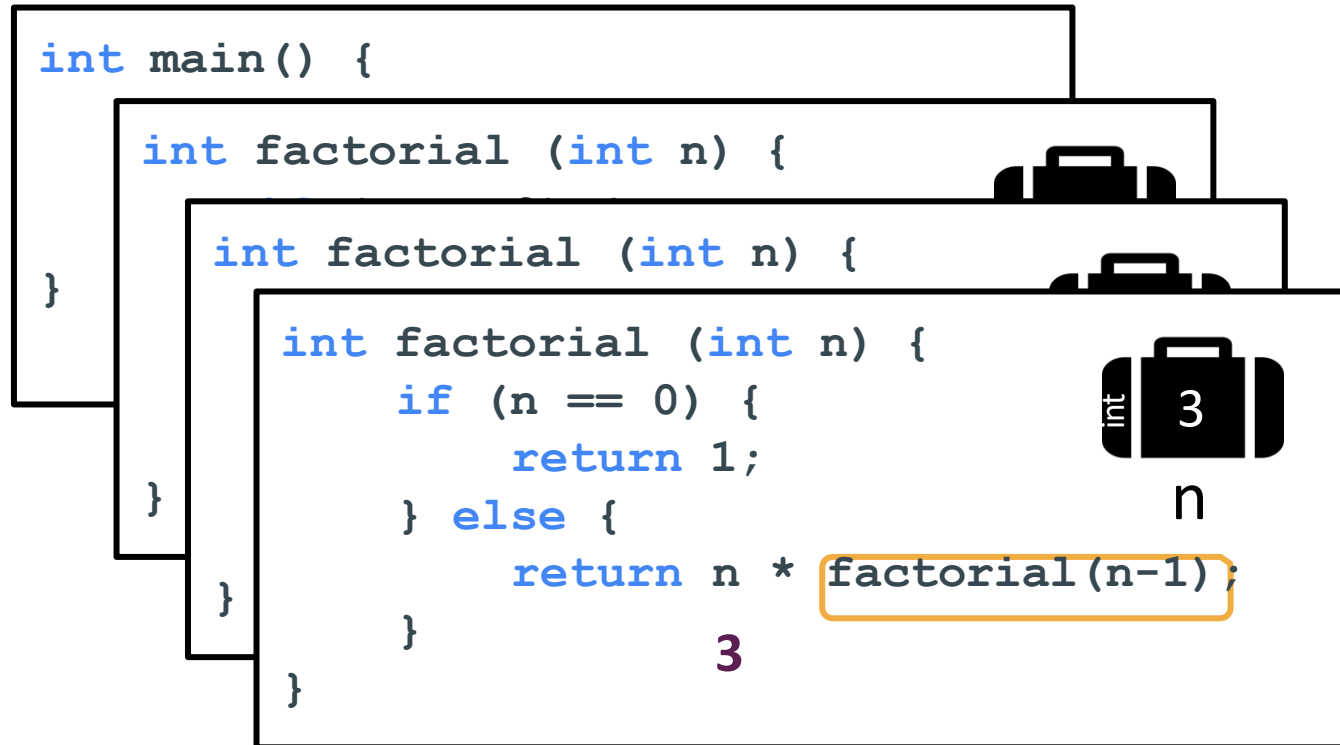
Recursion in action



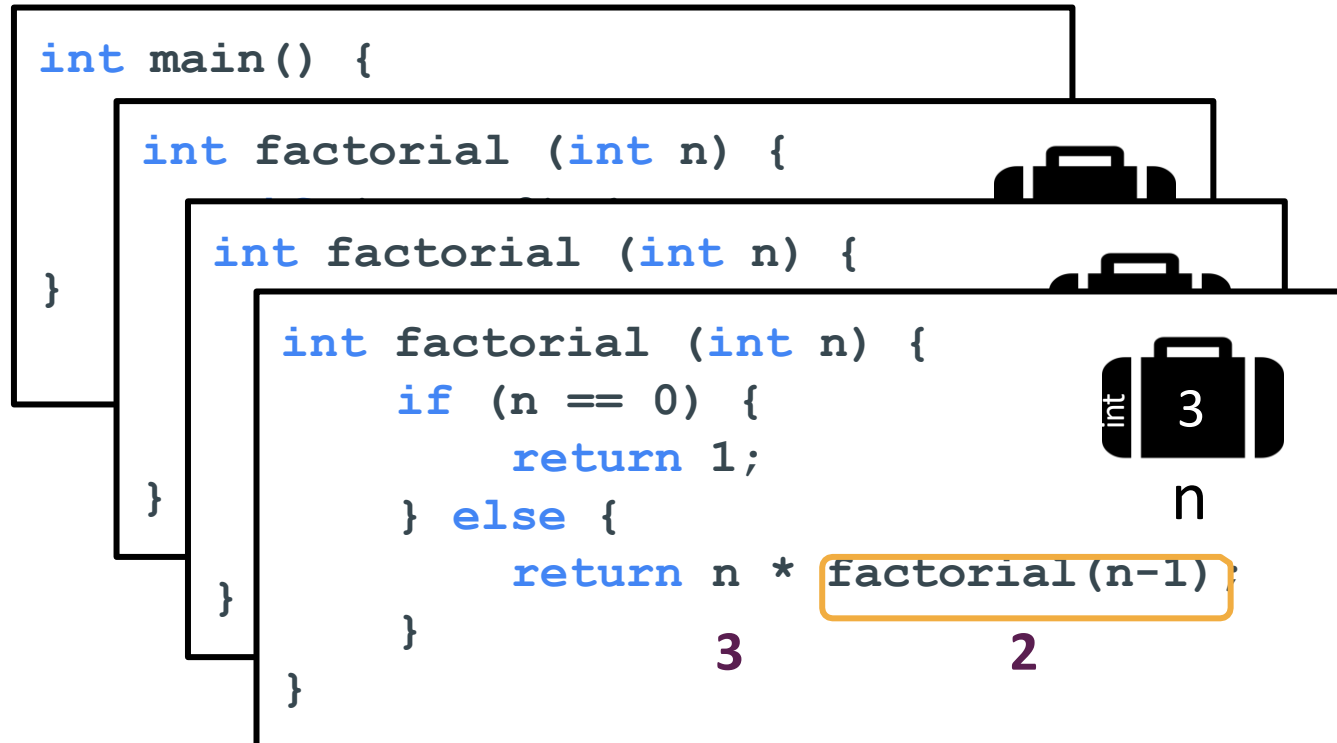
Recursion in action



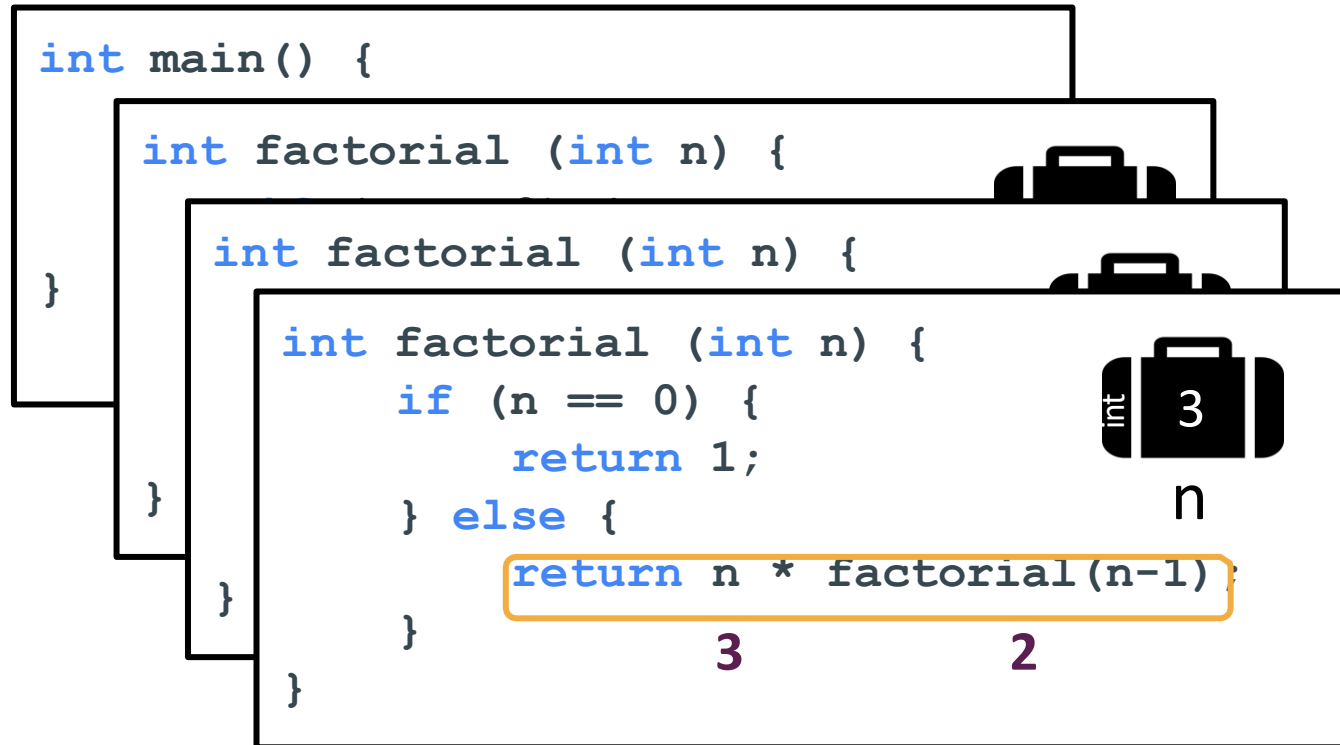
Recursion in action



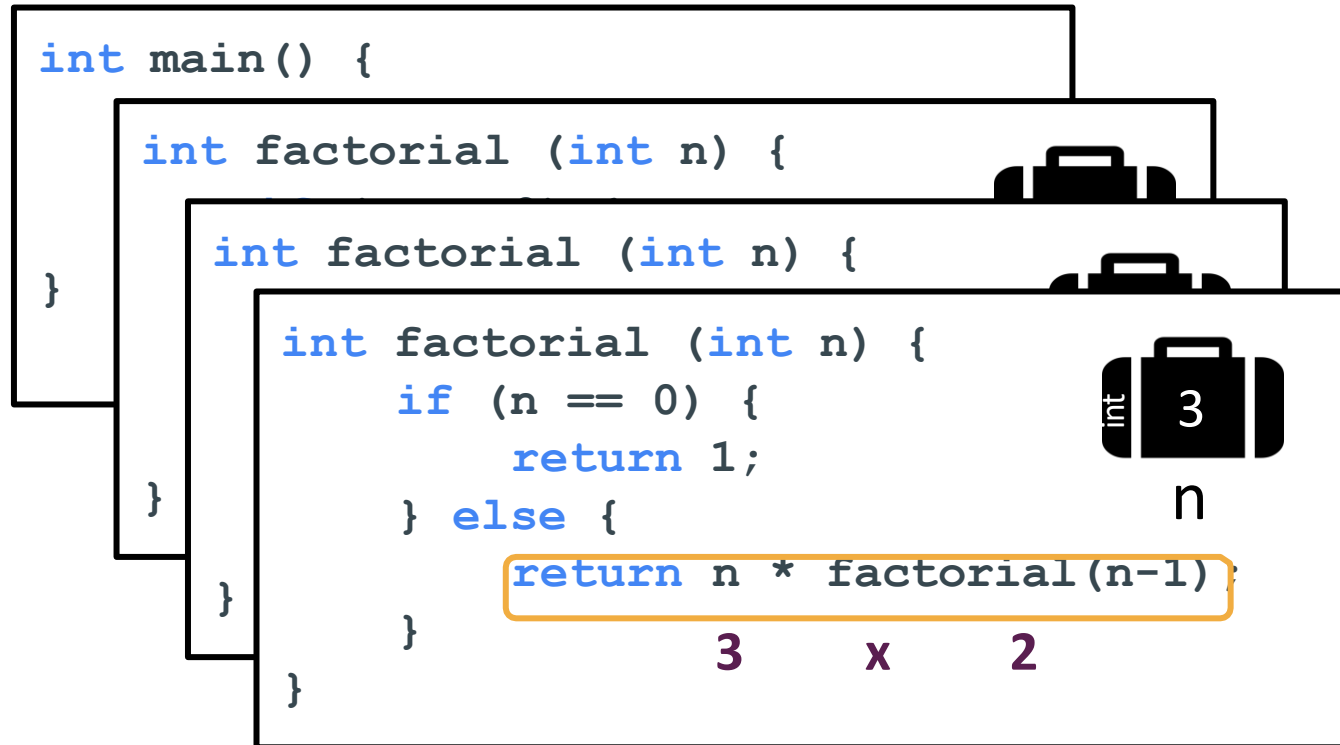
Recursion in action



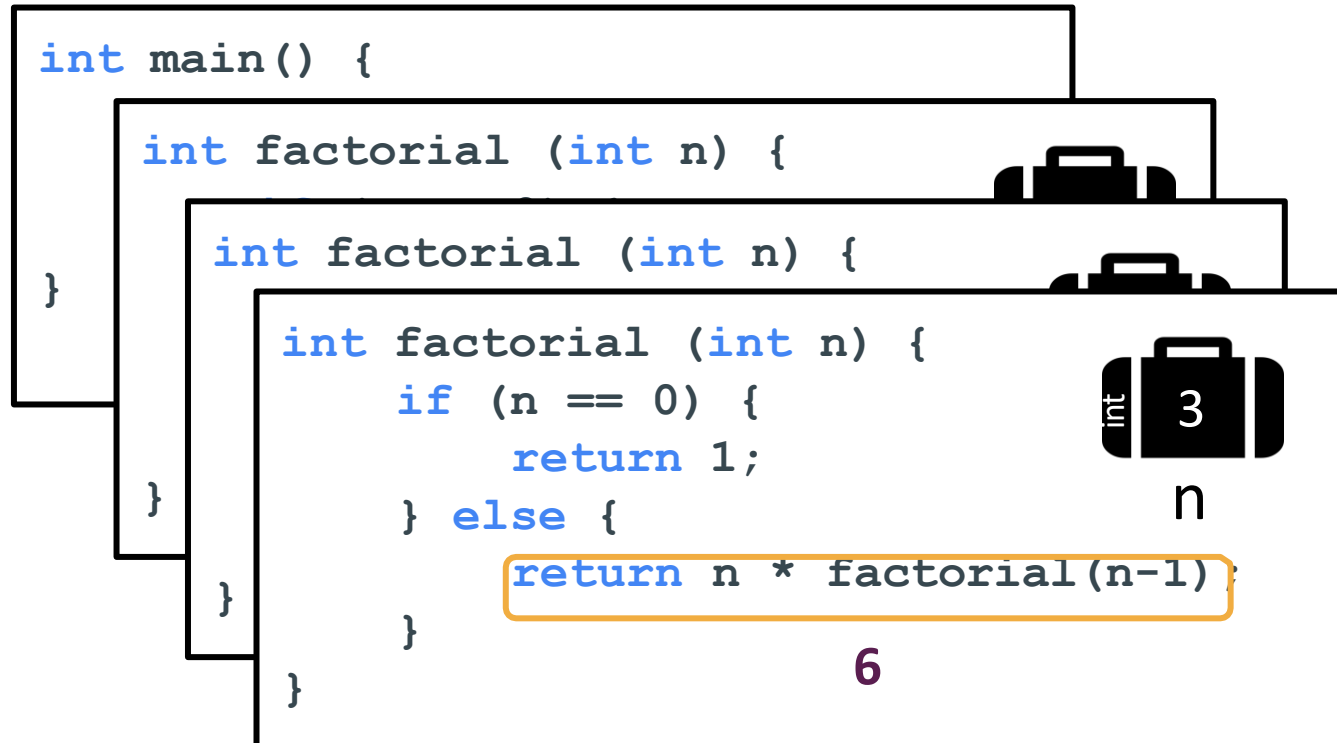
Recursion in action



Recursion in action



Recursion in action



Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

4

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

4

6

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

4

6

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

4

x

6

Recursion in action

```
int main() {
```

```
    int factorial (int n) {
```

```
    }
```

```
        int factorial (int n) {
```

```
            if (n == 0) {
```

```
                return 1;
```

```
            } else {
```

```
                return n * factorial(n-1);
```

```
            }
```

```
        }
```



n

24

Recursion in action

```
int main() {
```

```
}
```

```
int factorial (int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n-1);
```

```
    }
```

```
}
```



n

5

Recursion in action

```
int main() {
```

```
}
```

```
int factorial (int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n-1);
```

```
    }
```

```
}
```



n

5

24

Recursion in action

```
int main() {
```

```
}
```

```
int factorial (int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n-1);
```

```
    }
```

```
}
```



n

5

24

Recursion in action

```
int main() {
```

```
}
```

```
int factorial (int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n-1);
```

```
    }
```

```
}
```



n

5

x

24

Recursion in action

```
int main() {
```

```
}
```

```
int factorial (int n) {
```

```
    if (n == 0) {
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n-1);
```

```
    }
```

```
}
```



n

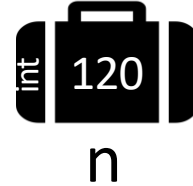
120

Recursion in action

```
int main() {  
    int n = factorial(5);  
    cout << "5! = " << n << endl;  
    return 0;  
}
```

Recursion in action

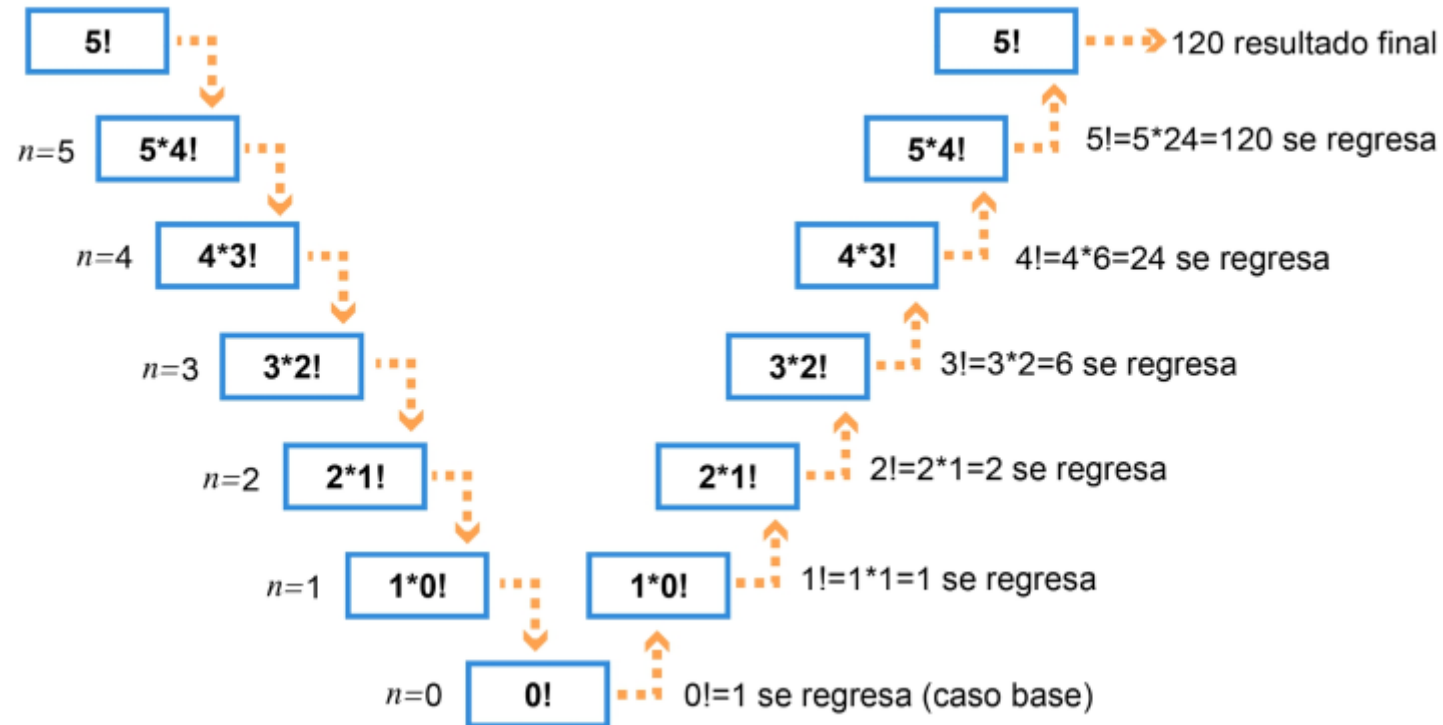
```
int main() {  
    int n = factorial(5);  
    cout << "5! = " << n << endl;  
    return 0;  
}
```



int 120
n

Recursion

Example factorial

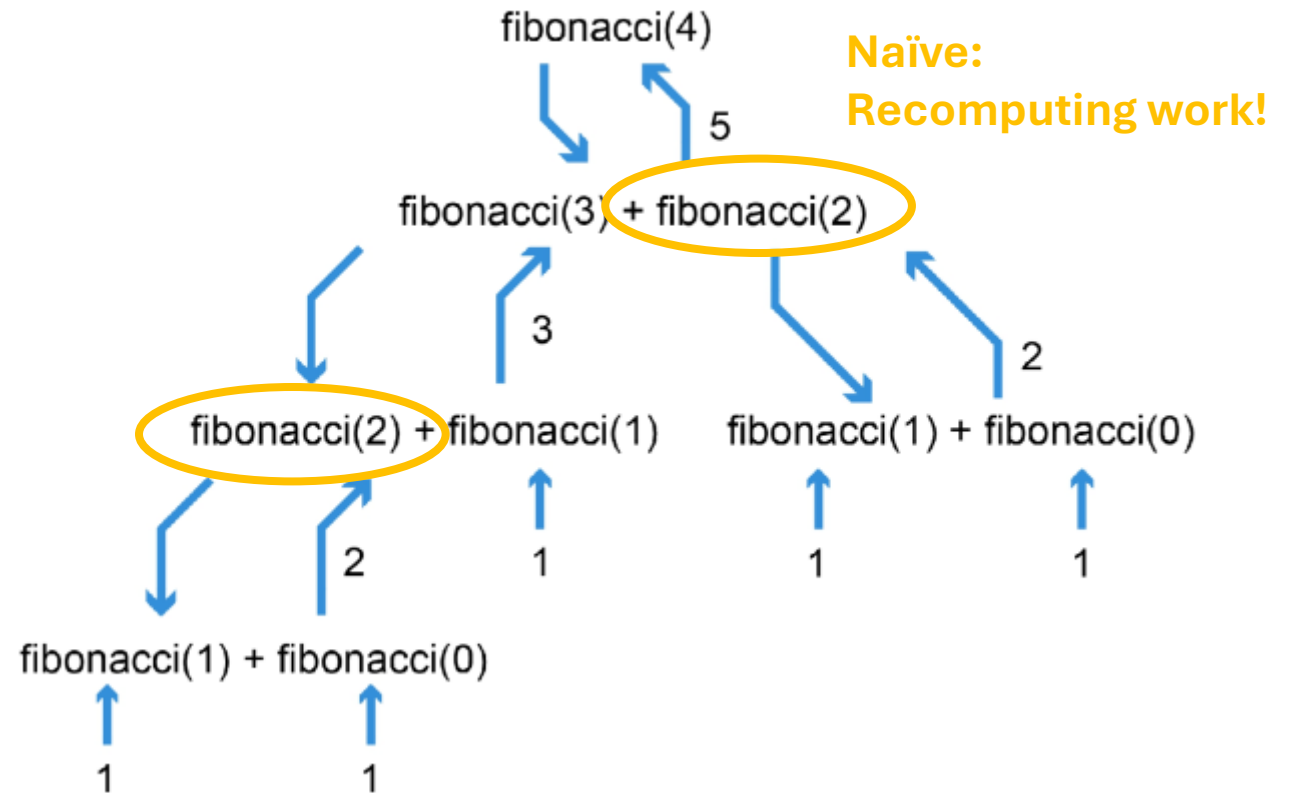


Recursion

Fibonacci – Naïve approach

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

```
1. long fibonacci(int n){  
2.     if (n==0 || n==1)  
3.         return 1;  
4.     else  
5.         return fibonacci(n-1)+fibonacci(n-2);  
6. }
```



Recursion

No progress → infinite recursion

- If a call never gets closer to a base case, the recursion never stops.
- It keeps calling itself until it runs out of stack space and crashes.
- There are no "special cases": every path must reach a base case.

For example:

Fibonacci(-5)



**IT WILL PRODUCE AN
STACK OVERFLOW**

```
1. long fibonacci(int n){  
2.     if (n==0 || n==1)  
3.         return 1;  
4.     else  
5.         return fibonacci(n-1)+fibonacci(n-2);  
6. }
```

Recursion

The 4 rules of recursion

1

Base cases

Always have at least one case solvable without recursion.

2

Making progress

Every recursive call must move toward a base case.

3

Design rule

Assume all recursive calls already work — the "leap of faith".

4

Compound interest rule

Never redo work by re-solving the same subproblem.

Recursion

Recursion vs Iteration

- Any recursive algorithm can be rewritten iteratively: recursion is a design choice, not a necessity.
- Recursion costs stack memory, one frame per pending call.
- The choice is a trade-off between clarity and stack memory.

Recursion

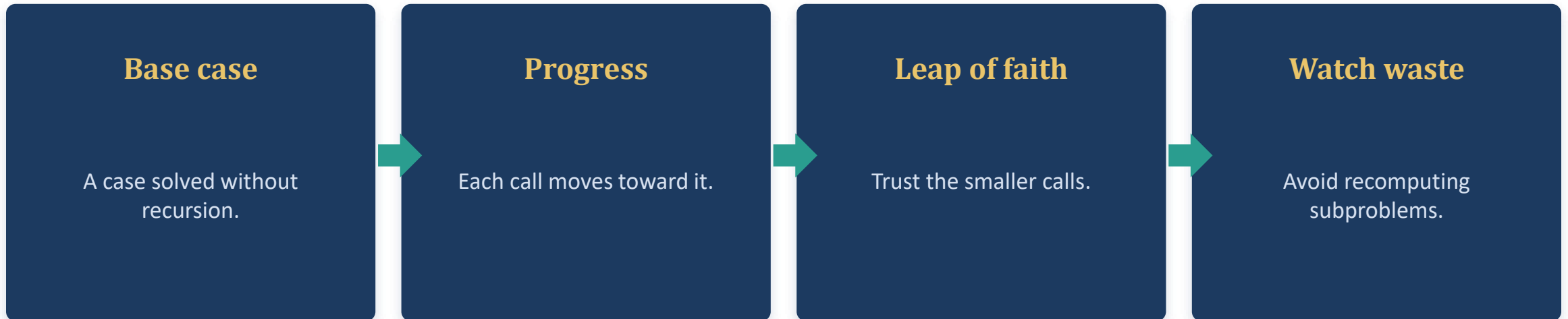
Recursion vs Iteration

```
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return n * factorial(n-1);  
    }  
}
```

```
int factorialIterative(int n) {  
    int result = 1;  
    for (int i = 1; i <= n; i++) {  
        result = result * i;  
    }  
    return result;  
}
```

Recap

Key ideas to keep



Takeaway: a recursive function = base case + progress; think in smaller subproblems.



Exercises

Exercise 1

Factorial

Write a function `long factorial(int n)` that takes one integer as a parameter. The function must recursively compute the factorial of the given number. You may assume the value passed as a parameter is an integer greater than or equal to zero.

Exercise 2

Fibonacci

Write a function `long fibonacci(int n)` that takes one integer as a parameter. The function must recursively compute the nth Fibonacci number using the naive recursive definition. You may assume the value passed as a parameter is an integer greater than or equal to zero.

Exercise 3

Power

Write a function `long power(int base, int n)` that takes two integers as parameters. The function must recursively compute a number raised to a power. The first parameter is the base and the second is the exponent. You may assume the exponent passed as a parameter is an integer greater than or equal to zero.

Exercise 4

Palindrome

Write the recursive function `bool palindrome(char str[], int i, int f)`, which takes three parameters. The first is an array of characters representing a string; the second and third are the array indices delimiting the portion of the string to check for being a palindrome. The first time this function is called, the second parameter is always 0 and the third is the length of the array minus one. The function returns true when the array passed as a parameter is a palindrome, and false otherwise.

References

M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed.
Boston, MA, USA: Pearson, 2014.

V. M. de la Cueva Hernández, L. H. González Guerra, y E. G. Salinas
Gurrión, *Estructuras de datos y algoritmos fundamentales*.
Monterrey, México: Editorial Digital del Tecnológico de Monterrey,
2020.